



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



SERLAB
Software Engineering Research

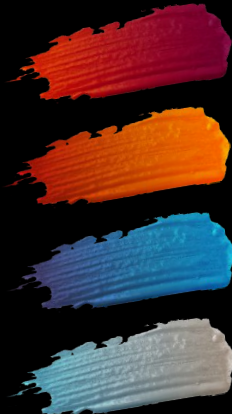
Ingegneria del Software

Processi Agili e SCRUM STREAM CODING

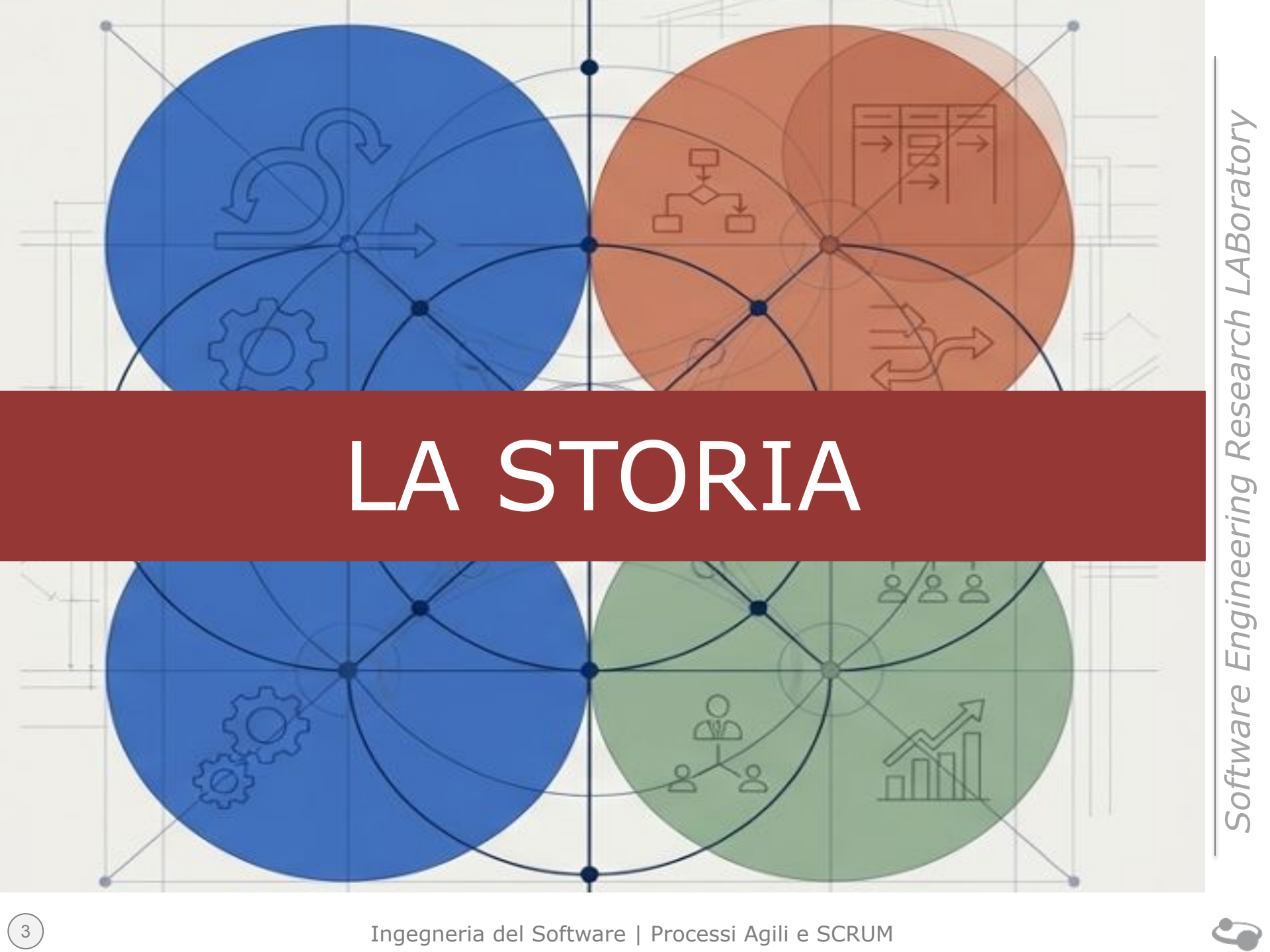
ITPS A.A. 2025/2026

Vita Santa Barletta
Danilo Caivano
Antonio Piccinno

Dipartimento di Informatica - Università degli Studi di Bari
Via Orabona, 4 - 70125 - Bari
Tel: +39.080.5443270 | Fax: +39.080.5442536
serlab.di.uniba.it







LA STORIA



Agile

- La sua origine è databile agli inizi del **2001** quando un pool di esperti di sviluppo software danno vita al famoso **Manifesto**, che ne definisce i Valori ed i Principi portanti
- Pensato come nuovo approccio in contrapposizione a quello che comunemente viene indicato come *document-driven software development*
- **Non definisce una metodologia specifica**, ma **un set di elementi comuni** che vanno a caratterizzare le sue diverse declinazioni come, ad esempio, Extreme Programming, Scrum, Adaptive software development, Feature driven development, ecc..
- Agile nasce specificatamente per il mondo dello sviluppo software



Sviluppo Agile con Metodi Agili

- I metodi agili sono progettati per produrre rapidamente software utile
- Sono metodi dello sviluppo incrementale, nei quali gli incrementi sono piccoli e, tipicamente, le nuove release del sistema vengono create e messe a disposizione dei clienti ogni due o tre settimane
- I clienti vengono coinvolti nel processo di sviluppo per ottenere un rapido feedback sulle modifiche dei requisiti
- La documentazione viene ridotta al minimo ricorrendo a comunicazioni informali, anziché a riunioni formali con documenti scritti



Agile Manifesto: i valori

Individui e interazioni	su	Processi e strumenti
Software funzionante	su	Documentazione esaustiva
Collaborazione col Cliente	su	Negoziazione di un contratto
Rispondere al cambiamento	su	Seguire un piano

Ciò che compare a sinistra è più importante.
Prevale, su ciò che c'è a destra



SCRUM: I Tre Pilastri Fondamentali

Questi tre elementi costituiscono il framework SCRUM: Trasparenza, Ispezione e Adattamento

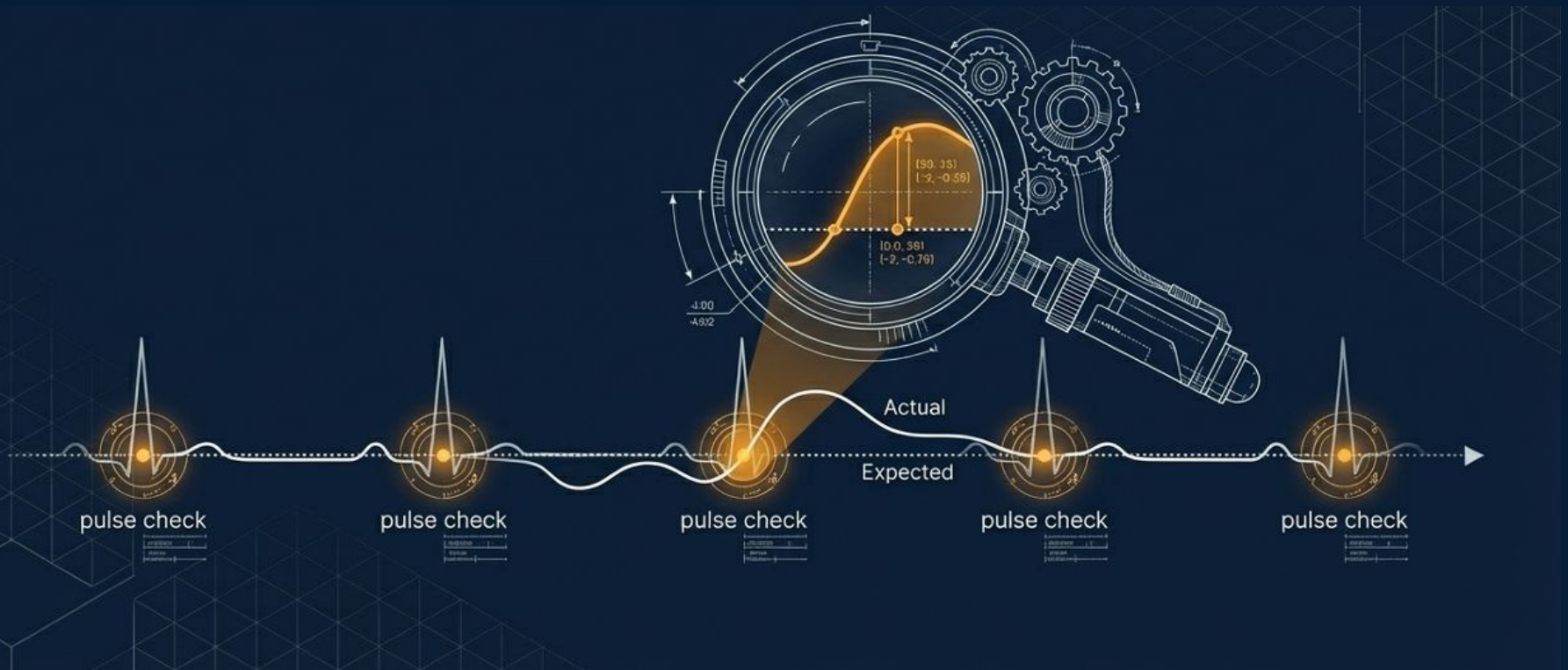


Software Engineering Research LABoratory

-

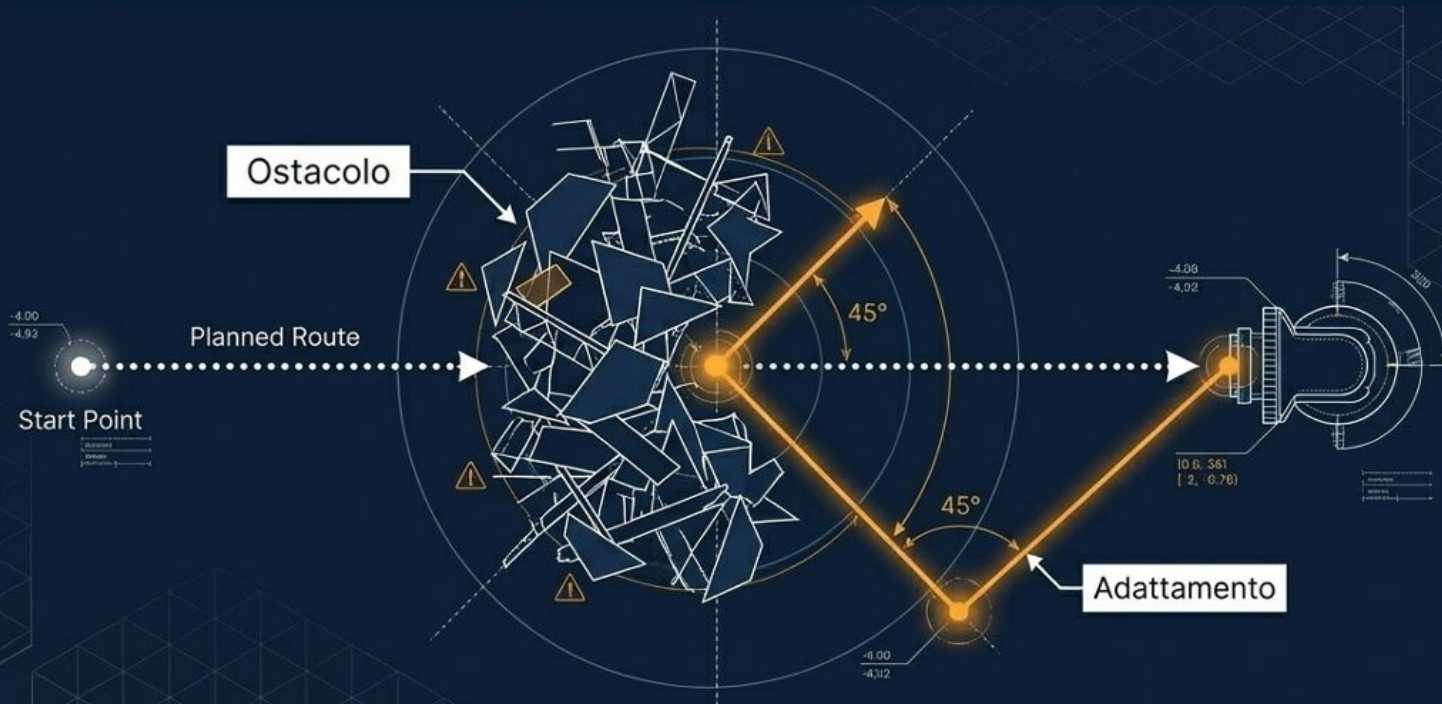
Pilastro 2: Ispezione

- Gli artefatti e i progressi devono essere ispezionati frequentemente per rilevare deviazioni indesiderate nel prodotto finale
- La pianificazione delle ispezioni deve essere sufficiente per intercettare i problemi, ma non deve intralciare il lavoro del team



Pilastro 3: Adattamento

- Se l'ispezione rivela che un aspetto del prodotto o del processo si allontana dai limiti accettabili, è necessario una correzione immediata per minimizzare ulteriori sprechi
- Ciò richiede un team autogestito e con l'autorità necessaria per agire



Agile Manifesto: I principi (1/2)

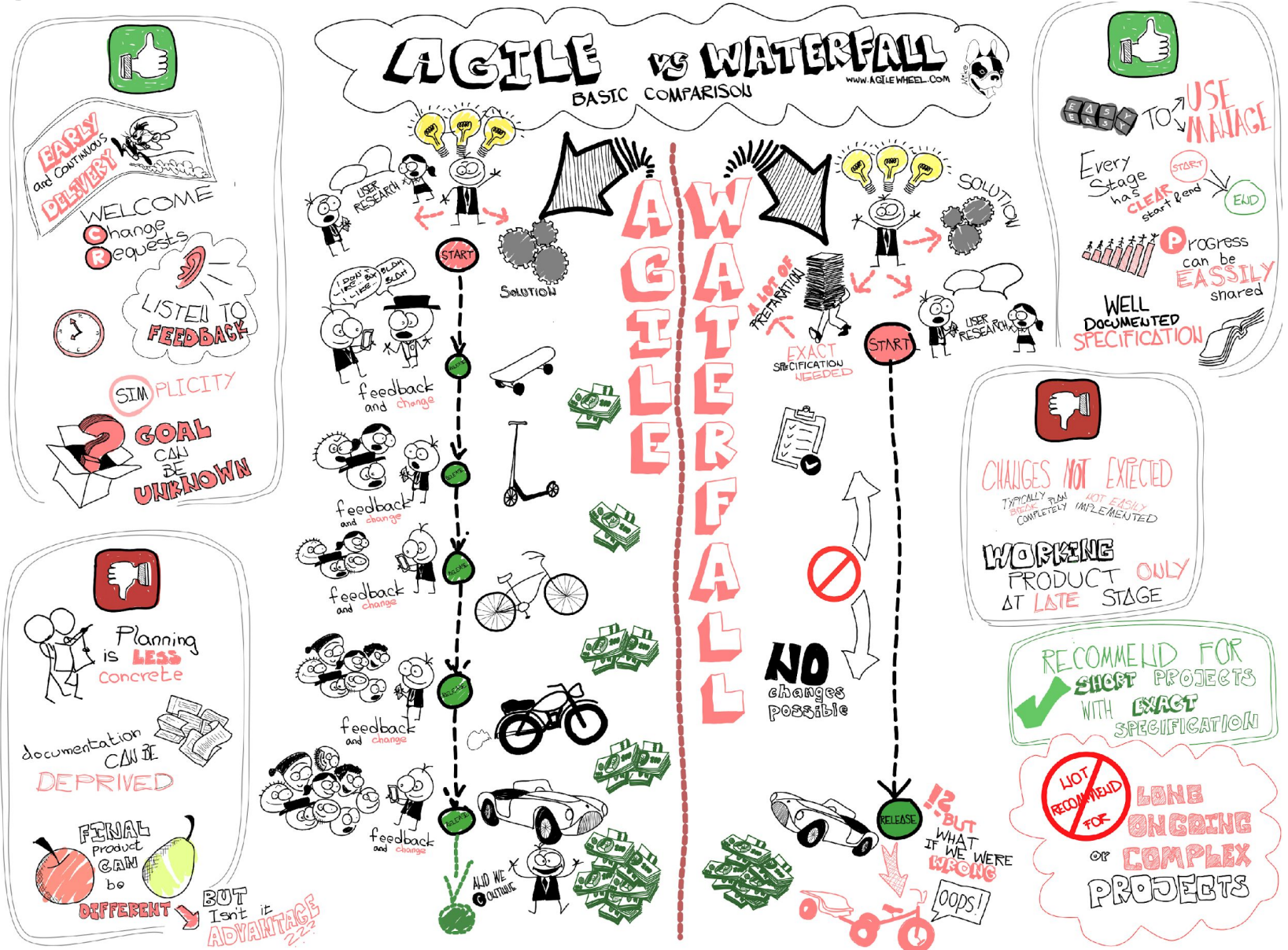
1. La nostra massima priorità è **soddisfare il cliente rilasciando software di valore**, fin da subito e in maniera continua
2. Accogliamo **i cambiamenti nei requisiti**, anche a stadi avanzati dello sviluppo. I processi agili sfruttano il cambiamento a favore del vantaggio competitivo del cliente
3. Consegniamo frequentemente **software funzionante**, con cadenza variabile da un paio di settimane a un paio di mesi, **preferendo i periodi brevi**
4. Committenti e sviluppatori devono **lavorare insieme quotidianamente** per tutta la durata del progetto
5. Fondiamo i **progetti su individui motivati**. Diamo loro l'ambiente e il supporto di cui hanno bisogno e confidiamo nella loro capacità di portare il lavoro a termine
6. Una **conversazione faccia a faccia** è il modo più efficiente e più efficace per comunicare con il team ed all'interno del team



Agile Manifesto: I principi (2/2)

7. Il **software funzionante** è il principale metro di misura di progresso
8. I processi agili promuovono uno **sviluppo sostenibile**. Gli sponsor, gli sviluppatori e gli utenti dovrebbero essere in grado di mantenere indefinitamente un ritmo costante
9. La continua attenzione **all'eccellenza tecnica** e alla **buona progettazione** esaltano l'agilità
10. La **semplicità** - l'arte di massimizzare la quantità di lavoro non svolto - **è essenziale**
11. Le architetture, i requisiti e la progettazione migliori emergono da **team che si auto-organizzano**
12. A intervalli regolari il **team riflette su come diventare più efficace**, dopodiché regola e adatta il proprio comportamento di conseguenza

Agile vs Waterfall





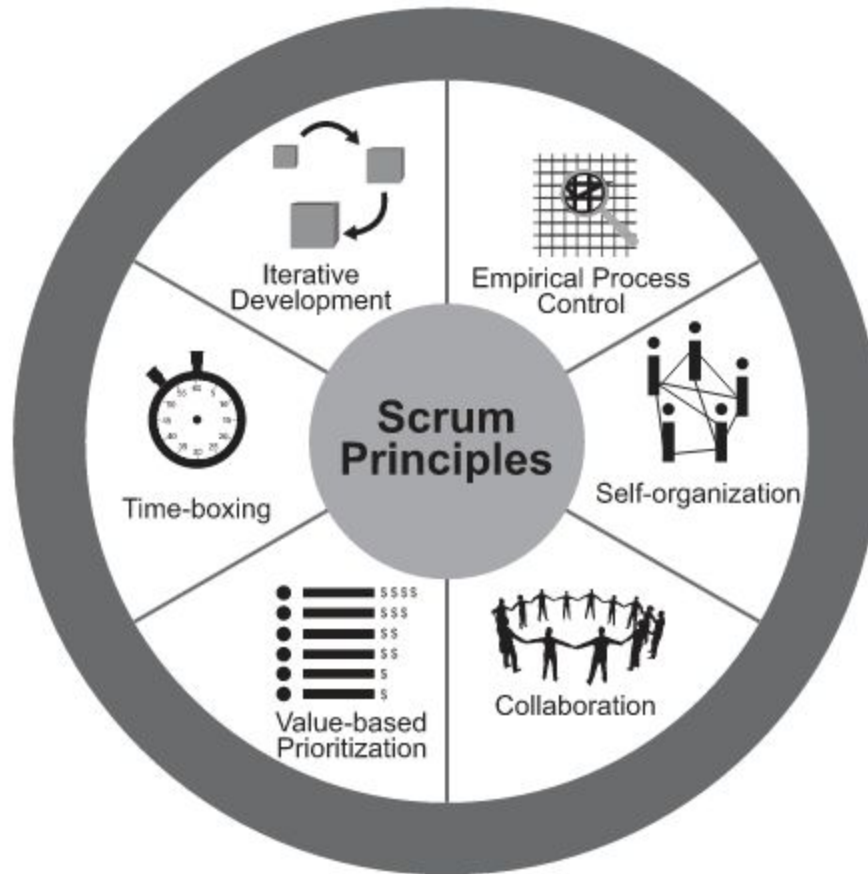
SCRUM



SCRUM - Alcune definizioni

- È uno tra gli approcci agili più diffusi ed utilizzati:
 - ❑ Non è semplicemente un processo o una tecnica per costruire prodotti ma **piuttosto è un framework** all'interno del quale è possibile utilizzare vari processi, strumenti e tecniche
 - ❑ Un framework che consente alle persone di **risolvere problemi complessi di tipo adattivo** e, al tempo stesso, di creare e rilasciare prodotti in modo efficace e creativo del più alto valore possibile
 - ❑ È un paradigma, un archetipo a cui potersi ispirare per produrre sistemi software complessi

Principi Guida



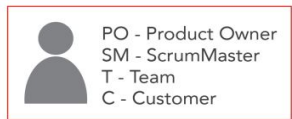
1. **Empirical Process Control** – Decisioni basate su conoscenza e continue sperimentazioni, piuttosto che su pianificazione di dettaglio apriori
2. **Self-Organization** – Lasciare che i gruppi di lavoro si autogestiscano ed auto-organizzino nello svolgimento del lavoro
3. **Collaboration** – Lo sviluppo del prodotto è uno sforzo collaborativo e un processo condiviso di creazione del valore
4. **Value-Based Prioritization** – Rilasciare il massimo valore nel più breve tempo possibile e pianificare in funzione di questo
5. **Time-boxing** – Il tempo è una risorsa limitata e, quindi, per ogni sviluppo deve essere predefinita e contingentata la quantità di tempo a disposizione
6. **Iterative Development** – I requisiti utente evolvono continuamente e lo sviluppo deve essere tale da poter accogliere i cambiamenti

Sprint

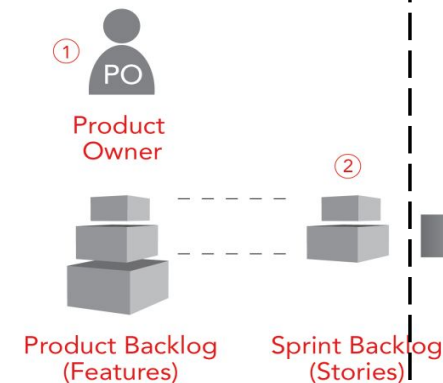
- Un progetto Scrum progredisce attraverso **una serie di "sprint"**
- Durata tipica è di 2-4 settimane o al massimo un mese
- In generale occorre pianificare la durata dello sprint in funzione di quanto tempo è possibile "rifiutare" una richiesta di un cambiamento
- Non ci deve essere **alcun cambiamento durante uno sprint**
- Una durata sempre costante porta a un miglior ritmo
- Il Prodotto è progettato, realizzato e testato durante lo sprint
- Uno sprint speciale è lo **"Sprint n.0"** nel quale va definito lo "Skeleton" ovvero la macro-architettura del sistema
- Lo **"Sprint n.0"** è l'unico che termina senza il rilascio di software funzionante



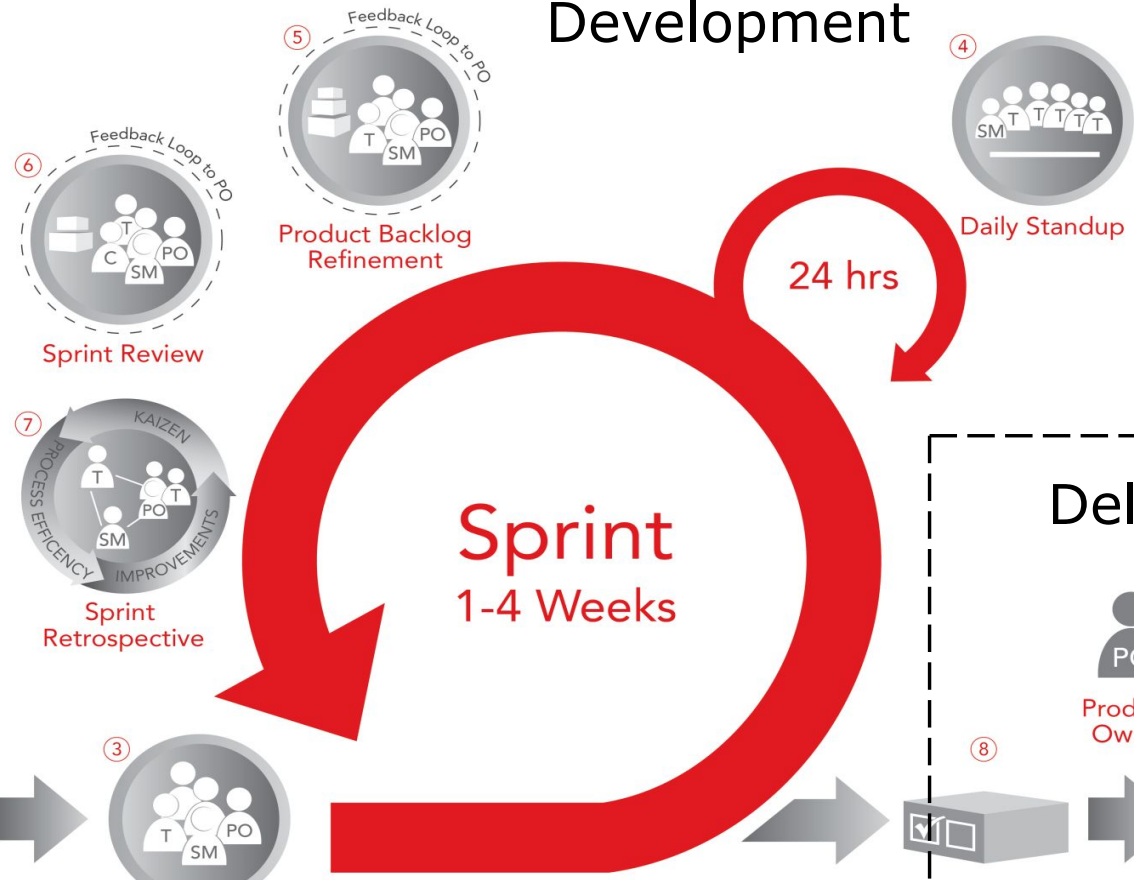
Overview



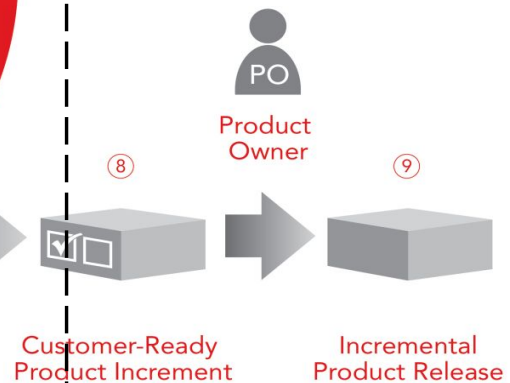
Inception



Development



Delivery



Quality Oriented Development

SONAR

ASANA

GRADLE

GIT

JENKINS

KIUWAN

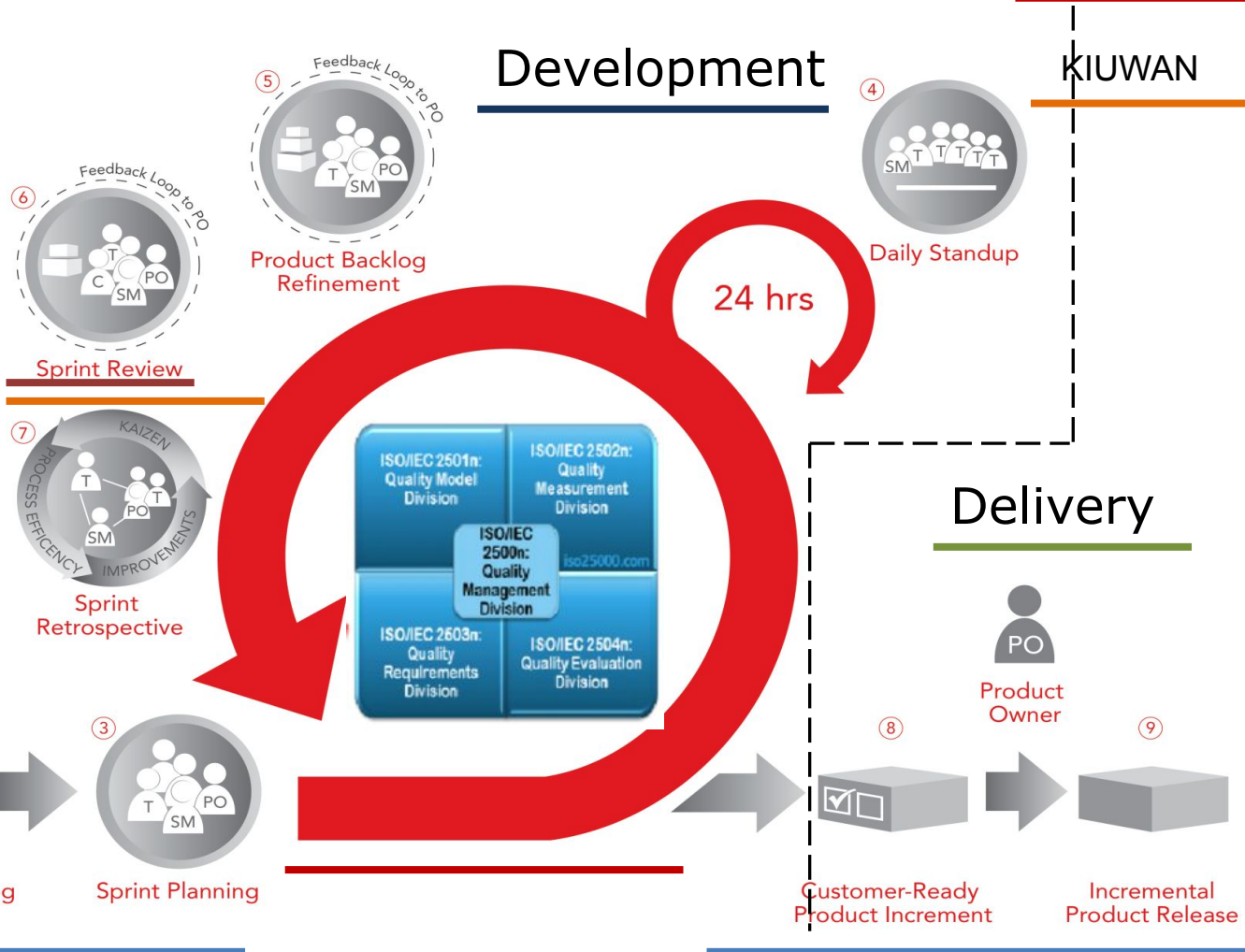


Inception



Product Backlog
(Features)

② Sprint Backlog
(Stories)

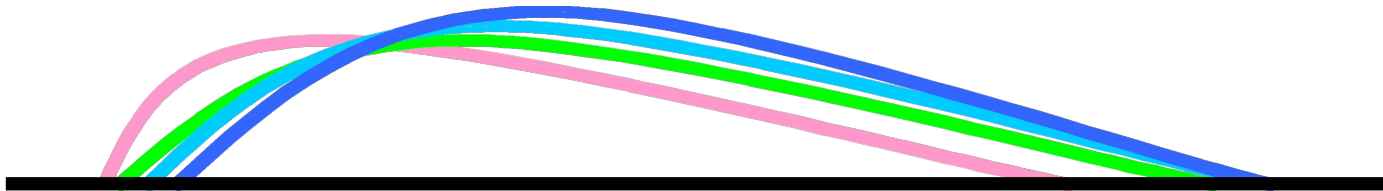


Sviluppo sequenziale vs parallelo



Piuttosto che completare una cosa alla volta...

... Con Scrum si fa un po' di tutto a ogni ciclo



"The New New Product Development Game" di Takeuchi and Nonaka. *Harvard Business Review*, Gennaio 1986.

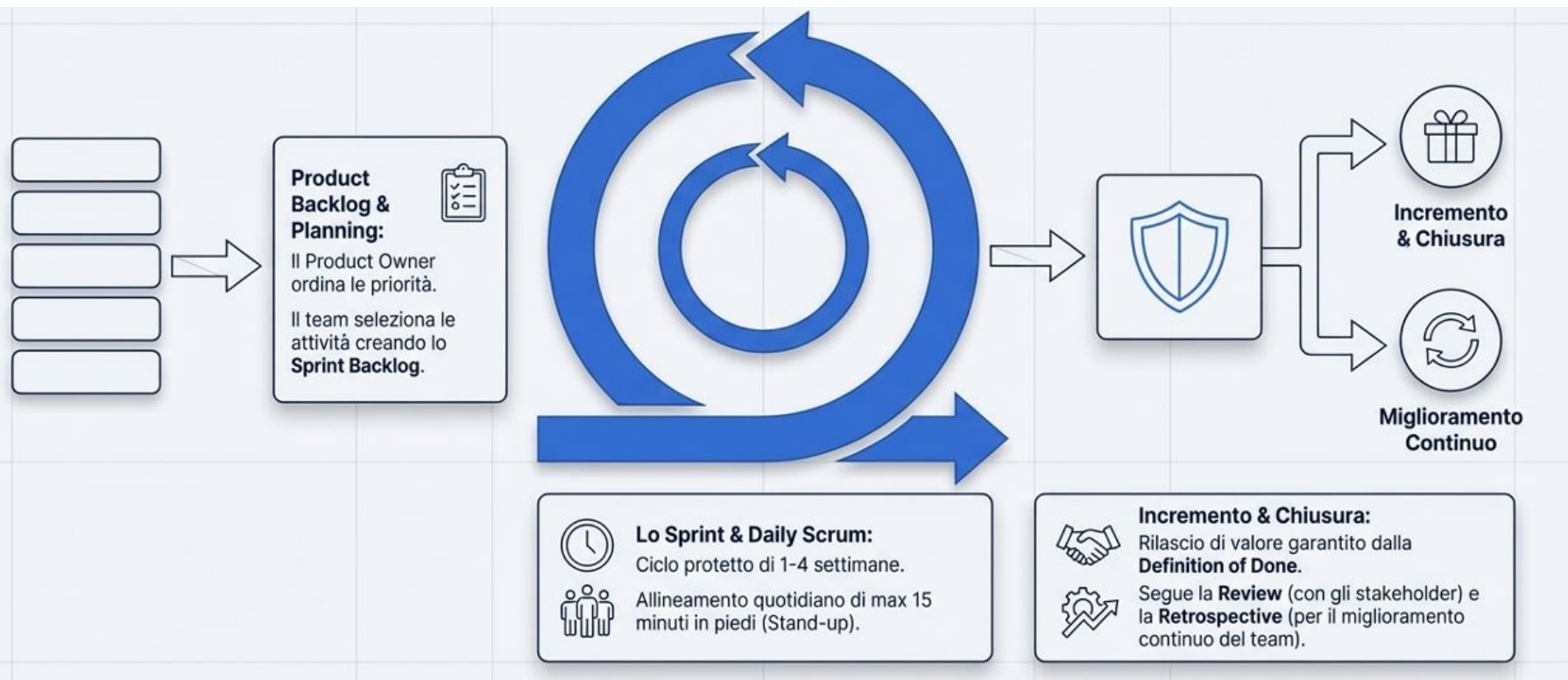


Predittivo vs Empirico

	Modello a Cascata (Predittivo)	Approccio Scrum (Empirico)
Conoscenza	Assunta all'inizio.	Emerge dall'esperienza.
Risposta al Cambiamento	Resistenza e controllo delle variazioni.	Adattamento continuo per massimizzare il valore.
Rilascio	"Big Bang" alla fine del progetto.	Incrementale e frequente.
Gestione del Rischio	Rischio altissimo fino al collaudo finale.	Rischio mitigato ad ogni Sprint.



L'anatomia del framework SCRUM



Cadenza

Sprint regolari e di durata fissa (1-4 settimane)

Filosofia del cambiamento

Nessuna modifica all'ambito durante lo Sprint in corso

Metriche Chiave

Velocità del team, Rilascio nei tempi pianificati, Software di qualità

Ideale per

Progetti complessi che richiedono rilasci programmati e apprendimento strutturato

Concetti Chiave

Ruoli

- Product owner
- ScrumMaster
- Team

Cerimonie

- Sprint planning
- Sprint review
- Sprint retrospective
- Daily scrum meeting

Strumenti

- Product backlog
- Sprint backlog
- Burndown charts
- Scrum Board



Ruoli

Ruoli

- Product owner
- ScrumMaster
- Team

Cerimonie

- Sprint planning
- Sprint review
- Sprint retrospective
- Daily scrum meeting

Strumenti

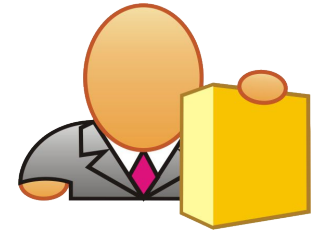
- Product backlog
- Sprint backlog
- Burndown charts
- Scrum Board



Software Engineering Research LABoratory



Product Owner



- Rappresenta la “committenza”
- Definisce le funzionalità del prodotto
- Definisce le priorità tra le funzioni in base al valore di mercato delle stesse
 - ❑ Decide le date di rilascio e il contenuto
- È responsabile della profittabilità (ROI) del prodotto
- Adatta le funzionalità dopo ogni ciclo
- Accetta o rifiuta i risultati del lavoro del team



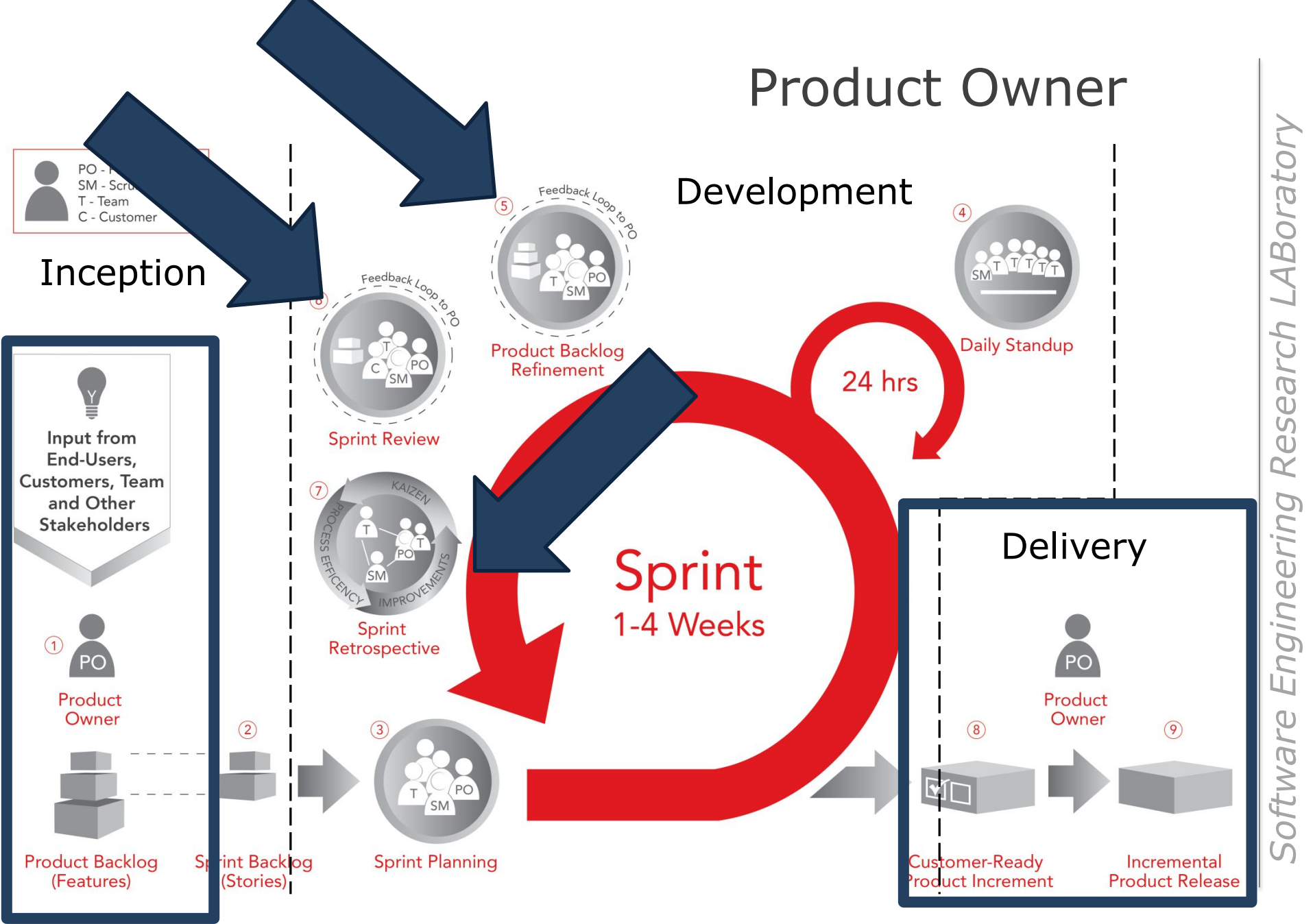
Product Owner

Development

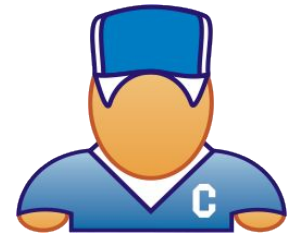
Inception

Delivery

Sprint 1-4 Weeks



ScrumMaster



- Rappresenta la funzione gestionale nel progetto
- Responsabile per la esecuzione e il miglioramento di pratiche e valori di Scrum
- Rimuove eventuali blocchi
- Assicura che il team sia pienamente operativo
- Facilita la collaborazione tra ruoli e funzioni
- Protegge il team dalle interferenze esterne





ScrumMaster

Developer

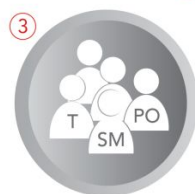
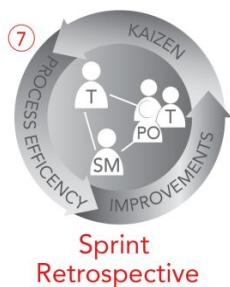
Initiation



Product Owner

Product Backlog (Features)

Sprint Backlog (Stories)



Sprint 1-4 Weeks

24 hrs



Delivery



Product Owner

Customer-Ready Product Increment

Incremental Product Release

Team

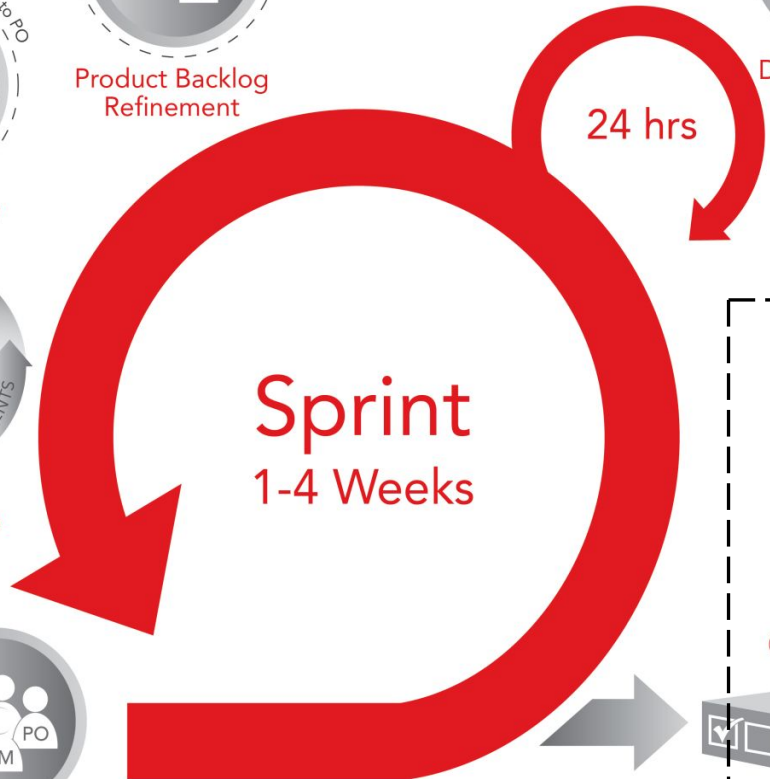
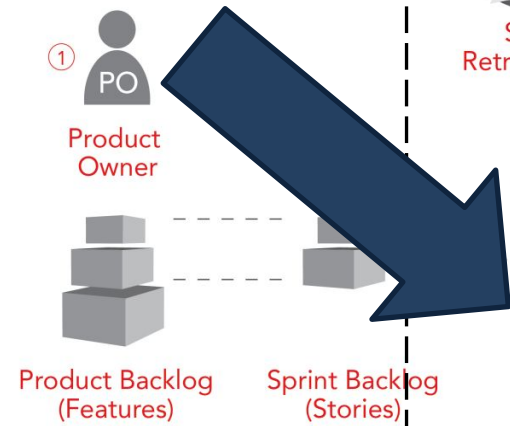
- Tipicamente 5-9 persone
- Interfunzionale:
 - ❑ Programmatori, tester, UX designer, etc.
- I membri sono generalmente allocati full-time
- È auto-organizzato
- I cambiamenti tra i membri sono possibili solo tra uno sprint e un altro (non durante uno sprint)



Team



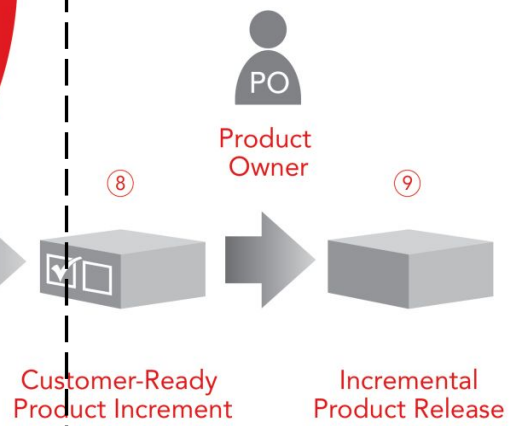
Initiation



Development



Delivery



Cerimonie

Ruoli

- Product owner
- ScrumMaster
- Team

Cerimonie

- Sprint planning
- Sprint review
- Sprint retrospective
- Daily scrum meeting

Strumenti

- Product backlog
- Sprint backlog
- Burndown charts
- Scrum Board



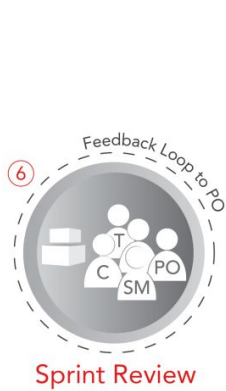
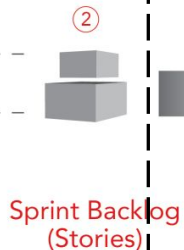
Sprint planning

PO - Product Owner
SM - ScrumMaster
T - Team
C - Customer

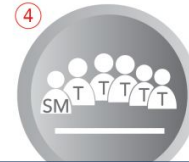
Inception
n



Product Owner



Development



Sprint
1-4 Weeks



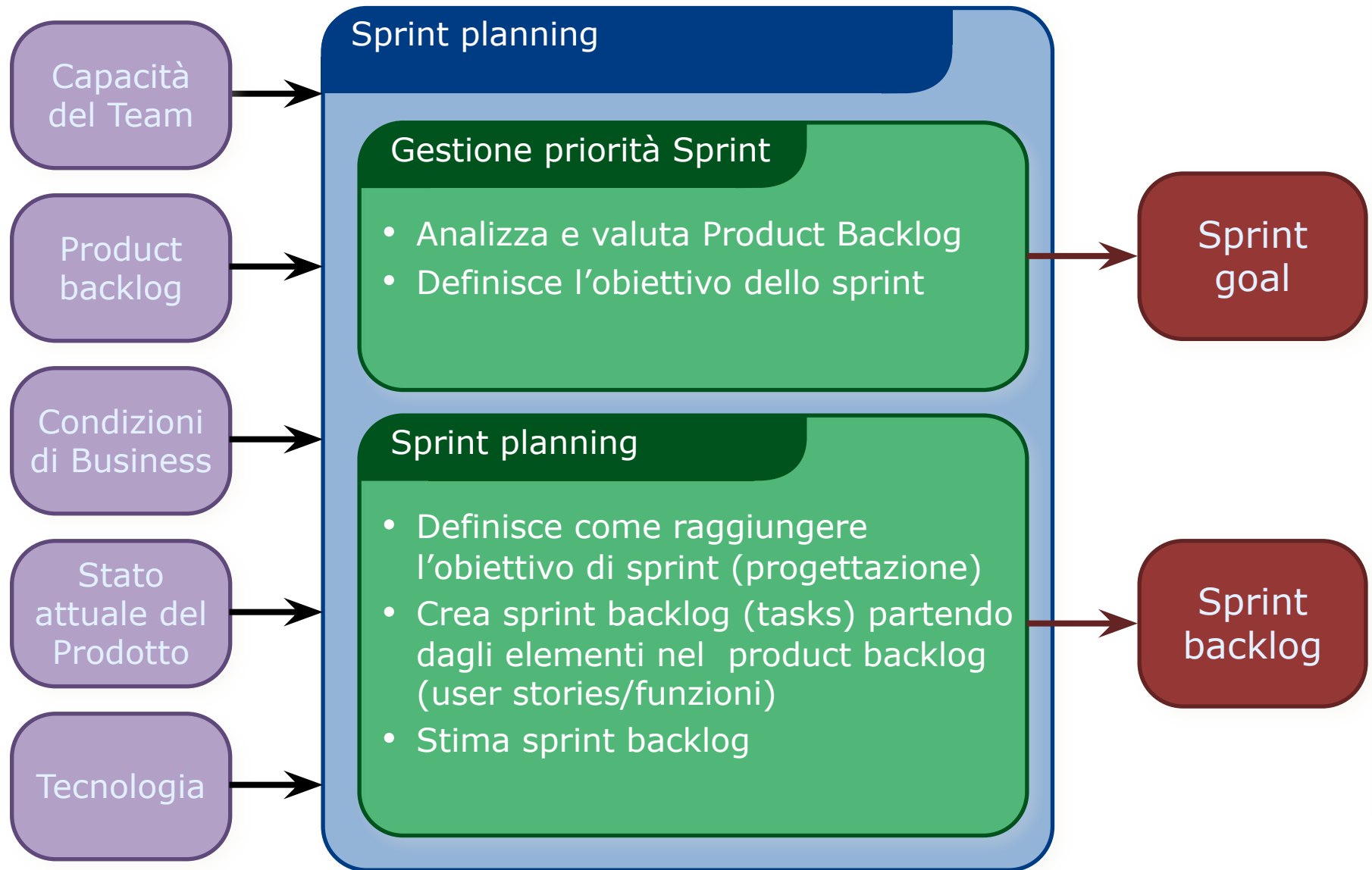
Customer-Ready Product Increment



Incremental Product Release



Sprint planning



Sprint planning

- Team seleziona elementi del product backlog che si impegna a completare nello sprint
- Si crea lo sprint backlog
 - ❑ Ogni task è definito e stimato
 - ❑ Tutti insieme, non solo lo ScrumMaster
- Considerare una progettazione ad alto livello

As a travel planner,
I want to see photos of
the hotels.



Middleware(8 ore)
UX (4 ore)
Test (4 ore)
Casi di esempio (6 ore)
Test prestazione (4 ore)

User Story (o Storia Utente)

- Permette di descrivere ed evidenziare l'importanza e l'impatto che una funzionalità avrà nel business
- Agevola la comunicazione con il cliente nell'esprimere l'utilità di una funzione del prodotto e la sua priorità
- Aiutano a non fare supposizioni
- Si focalizza sul descrivere cosa fare, per chi e perché deve essere fatto

User Story

- COME <ruolo>
- VOGLIO <fare qualcosa>
- COSÌ CHE <possa ottenere valore per il business>

Come magazziniere *--- ruolo*
voglio poter filtrare l'archivio ordini secondo *--- obiettivo*
la data di ricezione
per consultare gli ordini evasi *--- motivazione*



Daily Scrum

PO - Product Owner
SM - ScrumMaster
T - Team
C - Customer

Inception



Product Owner

Product Backlog (Features)

Sprint Backlog (Stories)

Development

Daily Scrum



Daily Standup

24 hrs



Sprint Review



Sprint Retrospective



Sprint Planning

Sprint
1-4 Weeks

Delivery



Product Owner

Customer-Ready Product Increment

Incremental Product Release



Daily scrum

- Come farlo
 - ❑ Ogni giorno
 - ❑ 15-minuti
 - ❑ In piedi
- Non è la sede dove si risolvono i problemi
 - ❑ Tutti possono partecipare
 - ❑ Solo i membri del Team, Scrum Master e Product Owner possono parlare
- Riduce riunioni non necessarie e il team ispeziona i progressi verso lo Sprint Goal e adatta lo Sprint Backlog per le successive 24 ore



Daily scrum

Cosa hai fatto ieri?

1

Cosa farai oggi?

2

Qualche impedimento?

3

- Non ha solo lo scopo di allineare i componenti del Team ma rappresenta anche un momento formale in cui ognuno prende in carico qualcosa da fare davanti a tutti gli altri



Product Backlog Refinement

PO - Product Owner
SM - ScrumMaster
T - Team
C - Customer

Inception



Product Owner

Product Backlog (Features)

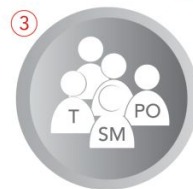
Sprint Backlog (Stories)



Sprint Review



Sprint Retrospective



Sprint Planning



Product Backlog Refinement

Sprint
1-4 Weeks

Product Backlog Refinement

Serve per eventualmente perfezionare a fine sprint il Product Backlog.
Non interviene il cliente

Delivery



Product Owner

Customer-Ready Product Increment

Incremental Product Release

Software Engineering

Product Backlog Refinement

- Team di sviluppo e Product Owner si riuniscono entro il termine dello Sprint per il **Product Backlog Refinement**
- Non è presente nelle cerimonie ufficiali
- Il refinement serve per mantenere il Product Backlog sempre aggiornato e con user story stimate in maniera appropriata a seconda delle priorità
- Lo scopo non è di passare in rassegna tutto il Product Backlog, ma **solo** quegli elementi che si prevede verranno inclusi nell'iterazione successiva o al massimo entro due Sprint

Sprint review



PO - Product Owner
SM - ScrumMaster
T - Team
C - Customer

Inception



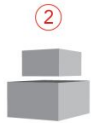
Input from
End-Users,
Customers, Team
and Other
Stakeholders



①
Product
Owner



Product Backlog
(Features)



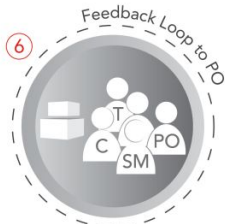
②
Sprint Backlog
(Stories)



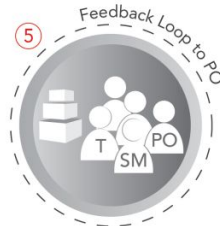
③
Sprint Planning



⑦
Sprint
Retrospective



⑥
Sprint Review



⑤
Product Backlog
Refinement

Development

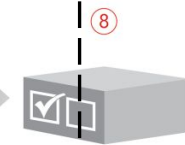


④

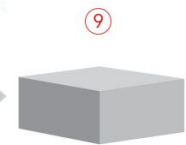
Sprint Review

Serve per verificare se e quali user stories completate a fine sprint possono considerarsi terminate, funzionanti e soprattutto rispettano le richieste del cliente

Sprint
1-4 Weeks



⑧
Customer-Ready
Product Increment



⑨
Incremental
Product Release

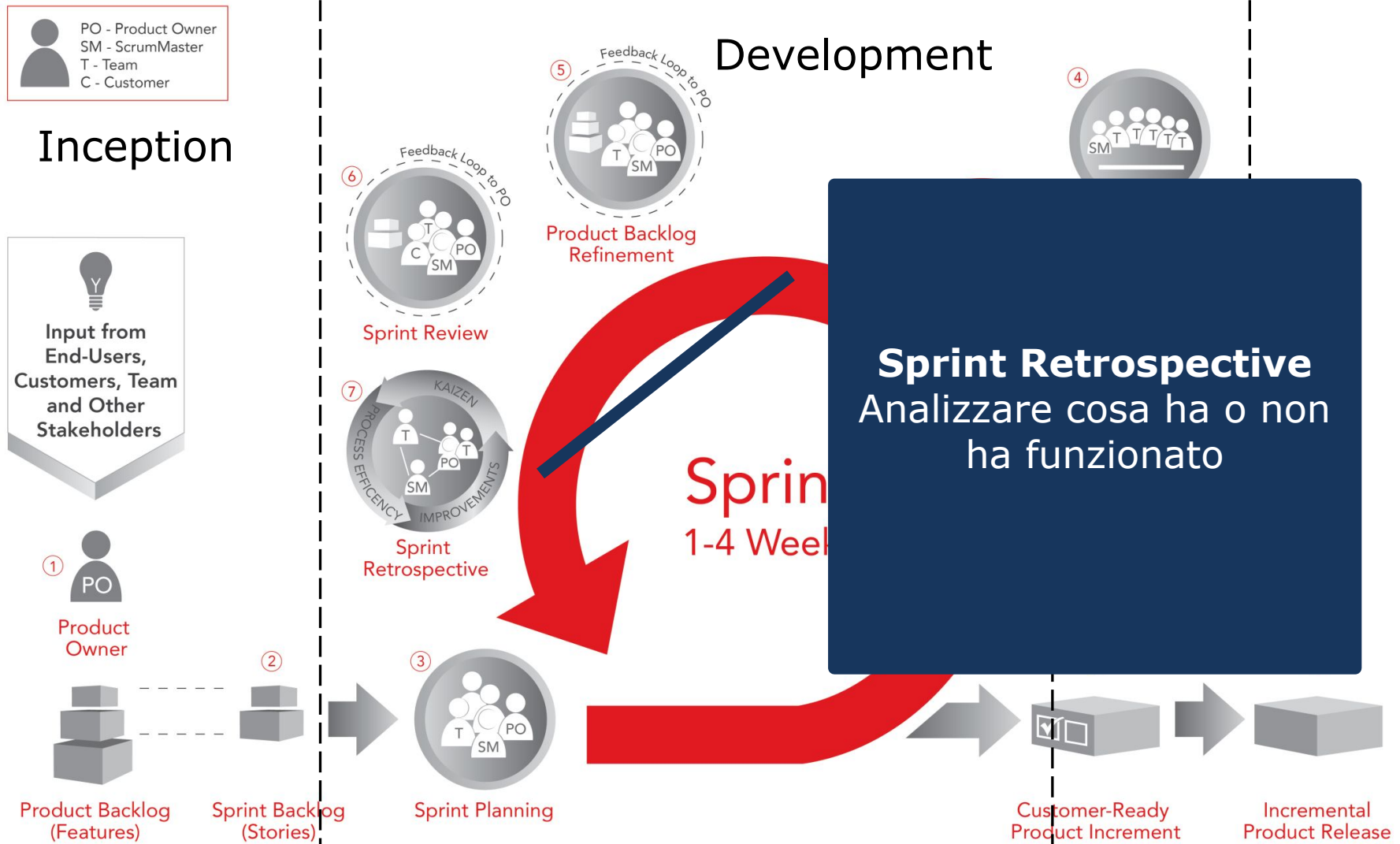


Sprint review

- Team presenta al Product Owner cosa ha realizzato durante lo sprint
- Tipicamente assume la forma di una demo delle funzionalità o della architettura dell'applicazione
- Informale
 - ❑ 2 ore
 - ❑ No slide ma demo
- Partecipa tutto il team
- Chiunque può essere invitato



Sprint Retrospective



Sprint retrospective

- Periodicamente bisogna analizzare cosa ha o non ha funzionato
- Tipicamente 15–30 minuti
- Eseguita alla fine di ogni sprint
- Partecipa tutto il team
 - ❑ ScrumMaster
 - ❑ Product owner
 - ❑ Team
 - ❑ Eventualmente clienti e altri

Cominciare a fare

Smettere di fare

Continuare a fare



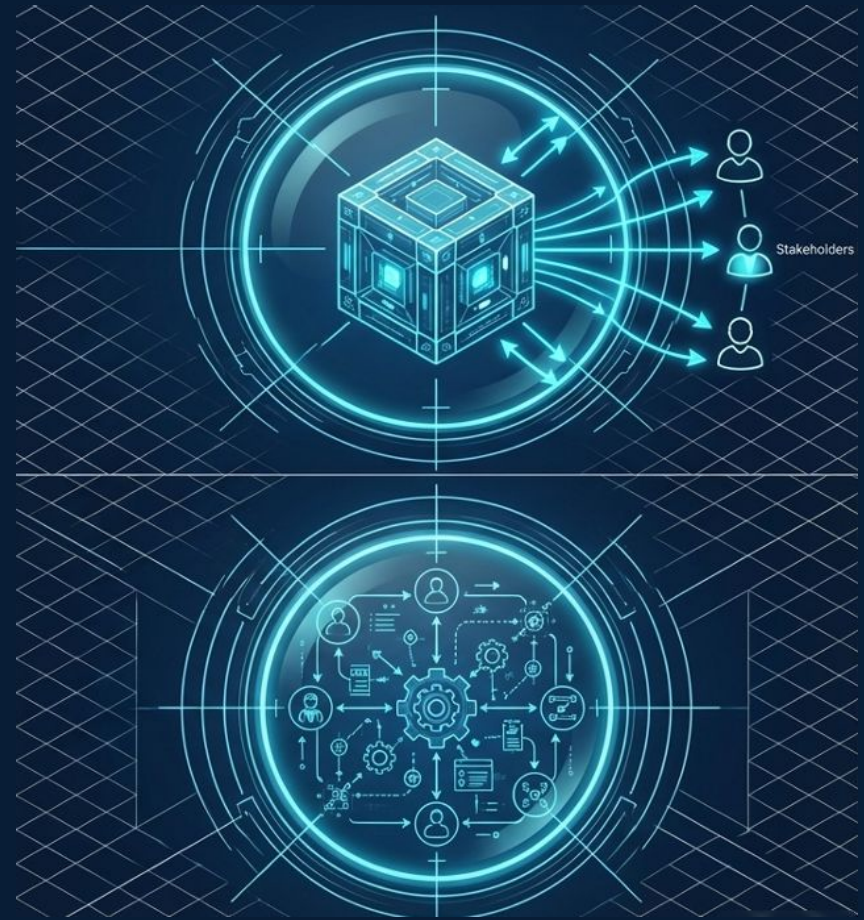
Sprint Review & Retrospective

Sprint Review: Focus sul Prodotto

Si ispeziona l'incremento con gli Stakeholder e si adatta il Product Backlog in base ai cambiamenti

Sprint Retrospective: Focus sul Processo e sulle Persone

Il team ispeziona se stesso, le interazioni e gli strumenti, pianificando adattamenti concreti per aumentare la qualità nello Sprint successivo



Strumenti

Ruoli

- Product owner
- ScrumMaster
- Team

Cerimonie

- Sprint planning
- Sprint review
- Sprint retrospective
- Daily scrum meeting

Strumenti

- Product backlog
- Sprint backlog
- Burndown charts
- Scrum Board



Product backlog

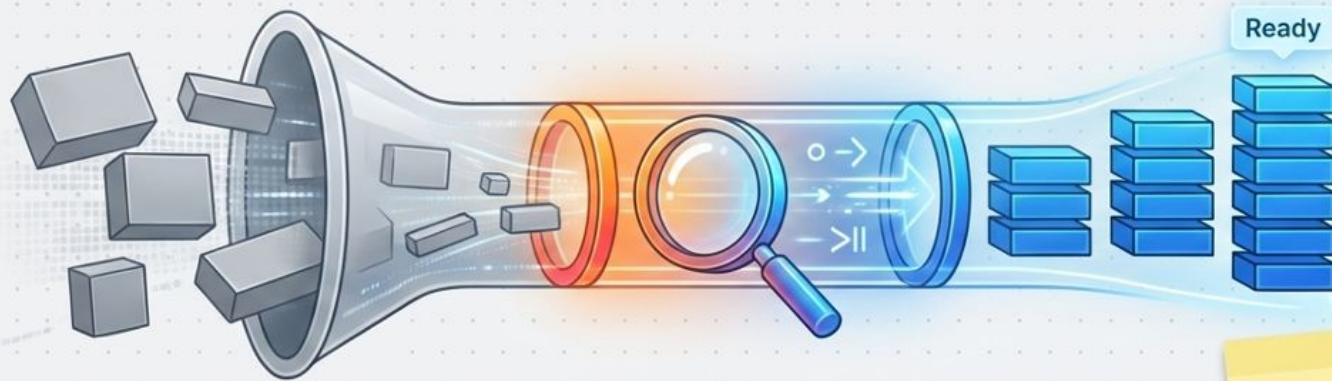
- Requisiti
- La lista di tutte le funzioni desiderate
- Idealmente costruito in modo che ogni elemento abbia un valore per i Clienti finali
- Priorità gestita dal Product Owner
- Priorità verificata all'inizio di ogni sprint

Elemento del backlog	Stima
Come ospite, voglio prenotare una camera	3
Come ospite, voglio cancellare una precedente prenotazione	5
Come ospite, voglio modificare le date di una precedente prenotazione	3
Come impiegato dell'hotel, voglio eseguire un report RevPAR reports (revenue-per-available-room)	8
Migliorare la gestione delle eccezioni	8
...	30
...	50



Product Backlog Refinement

Scomporre e definire ulteriormente gli elementi del Product Backlog aggiungendo descrizione, ordine e dimensione



Consuma in media il 5-10% della capacità dello Sprint del team.



Risponde alle domande con il PO (il Perché) e i Developer (il Come e la Stima).

Focus Operativo:
Le User Story devono essere pronte prima del Planning, non definite durante.

Sprint goal

- Una frase che indica su cosa ci si deve focalizzare durante lo sprint

Applicazione Database

Una applicazione che può eseguire comandi SQL su un DB Oracle.

Biologia

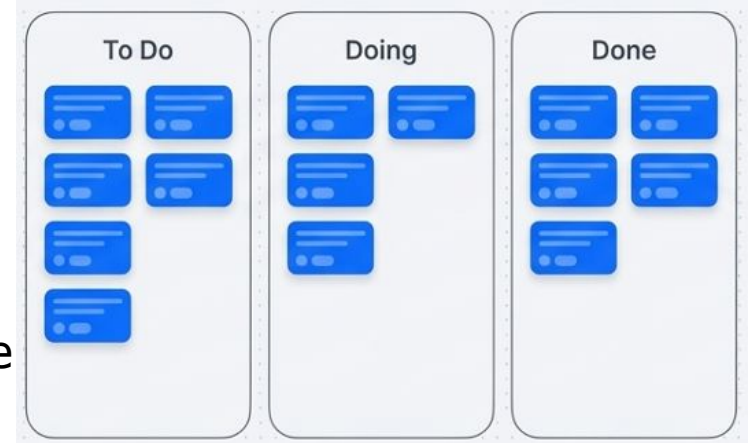
Funzioni di supporto per permettere studi di genetica.

Servizi finanziari

Integrare indicatori real time sulla compagnia ABC.



Sprint backlog



- Ognuno sceglie il lavoro da realizzare
 - ❑ Il lavoro **NON** viene assegnato
- La stima del lavoro rimanente viene aggiornata quotidianamente
- Ogni membro può migliorare i requisiti presenti nello sprint backlog
- Il lavoro sulle storie dello sprint “emerge”
- Se un requisito non è chiaro all’inizio dello sprint utilizzare il lavoro nello sprint per chiarirlo
- Aggiornare il lavoro restante mano a mano che i requisiti diventano più chiari

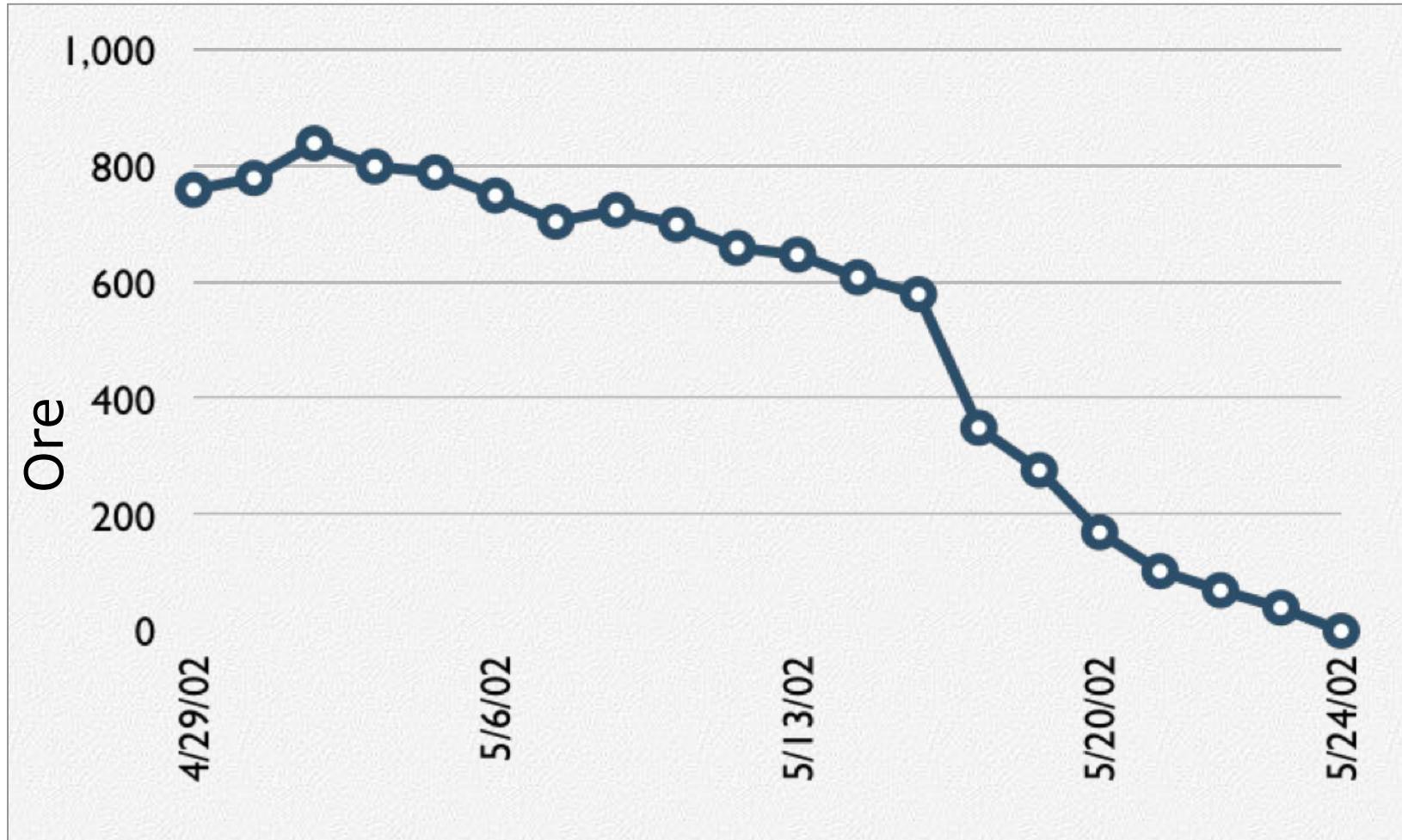
Un esempio di sprint backlog

Tasks	Lun	Mar	Mer	Gio	Ven
Realizzare interfaccia UI	8	4	8		
Realizzare Middleware	16	12	10	4	
Testare middleware	8	16	16	11	8
Scrivere help online	12				
Scrivere classi di esempio	8	8	8	8	8
Tracciare errori sul log			8	4	

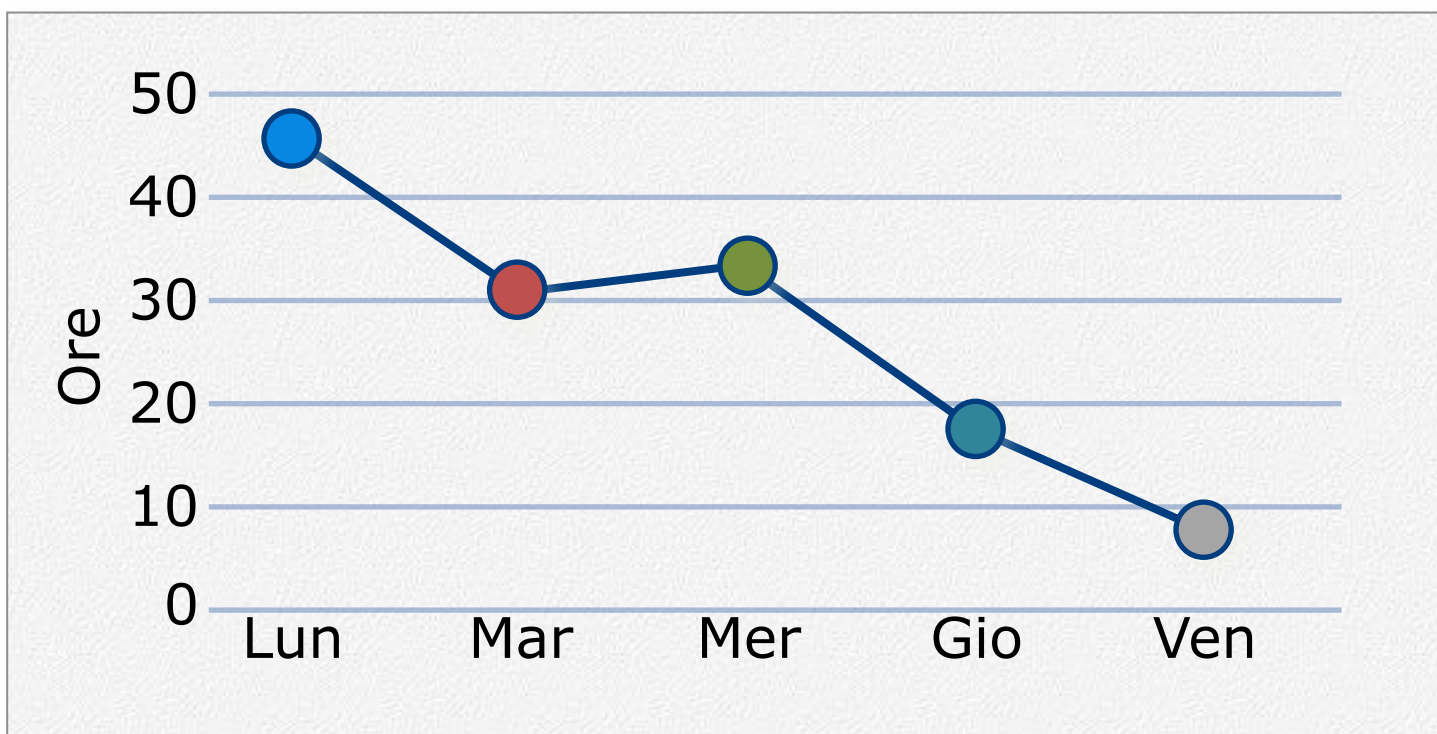
Sprint Burndown Chart

- Rende visibile il lavoro del Team
- È una rappresentazione grafica che mostra la velocità con cui il lavoro è completato e quanto lavoro rimane da fare
- È uno strumento efficace di reporting che mostra i progressi del Team verso lo Sprint Goal, non in termini di tempo speso ma in termini di quanto lavoro rimane
- Se il trend mostrato nel grafico non tende verso il basso già a partire da metà Sprint, occorre prendere provvedimenti

Un esempio di sprint burndown chart

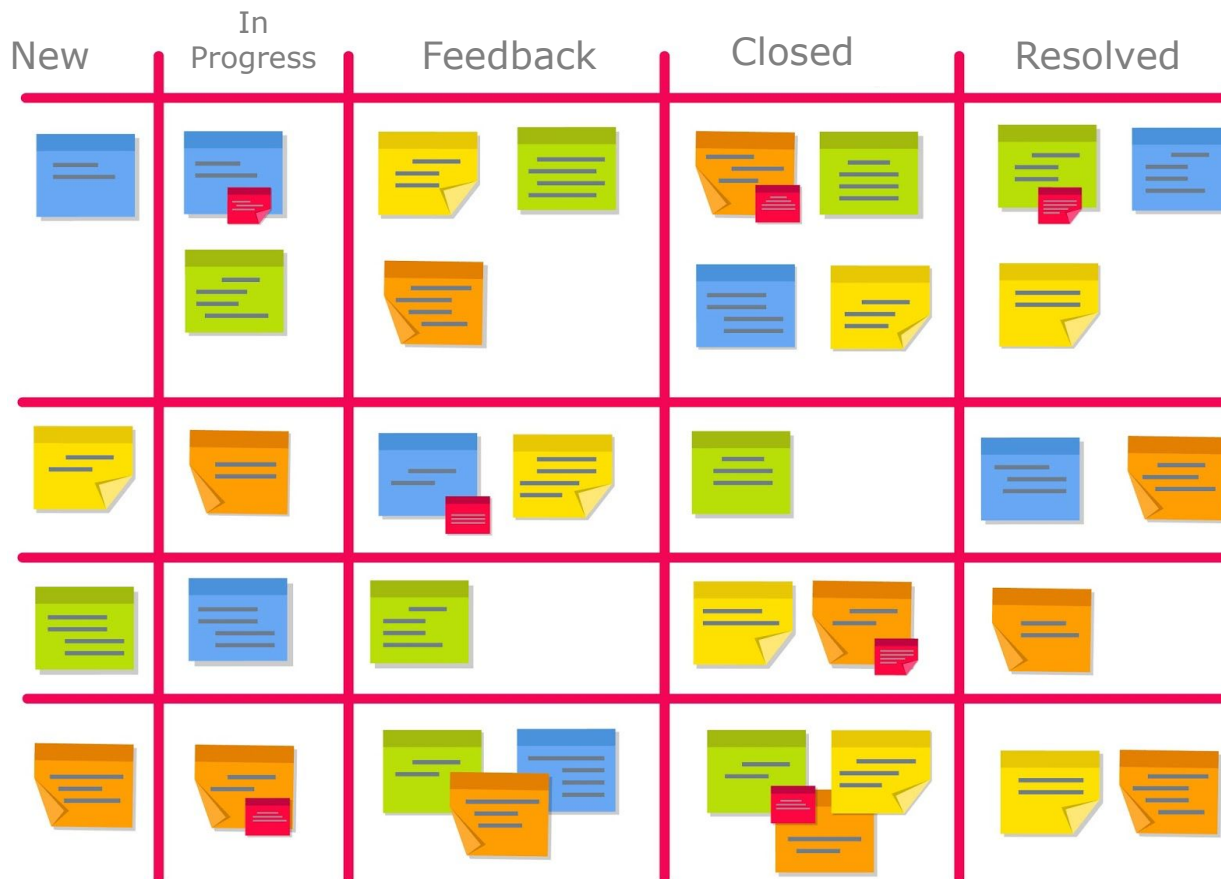


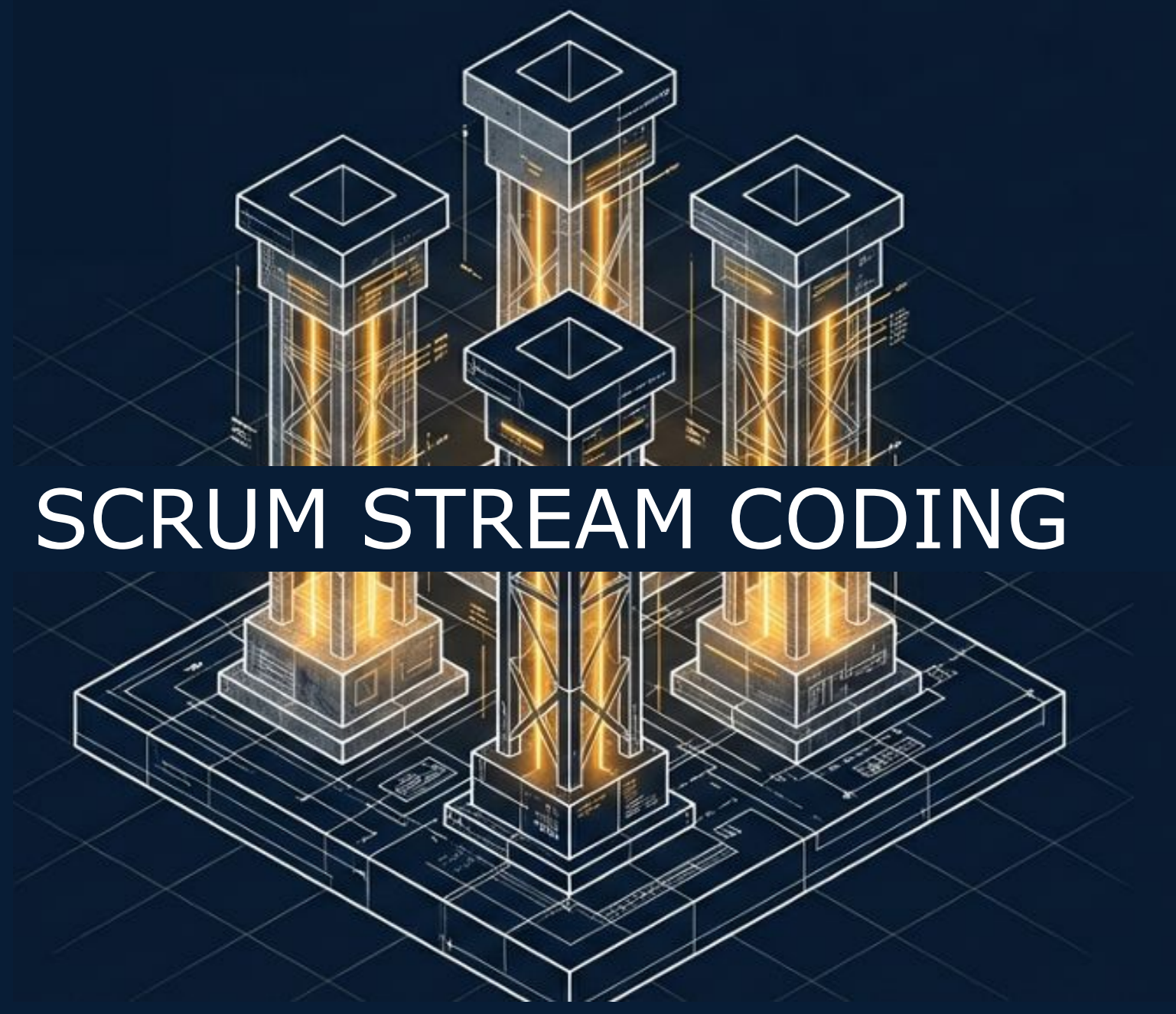
Attività	Lun	Mar	Mer	Gio	Ven
Realizzare interfaccia UI	8	4	8		
Realizzare Middleware	16	12	10	7	
Testare Middleware	8	16	16	11	8
Scrivere help online	12				



Scrum Board

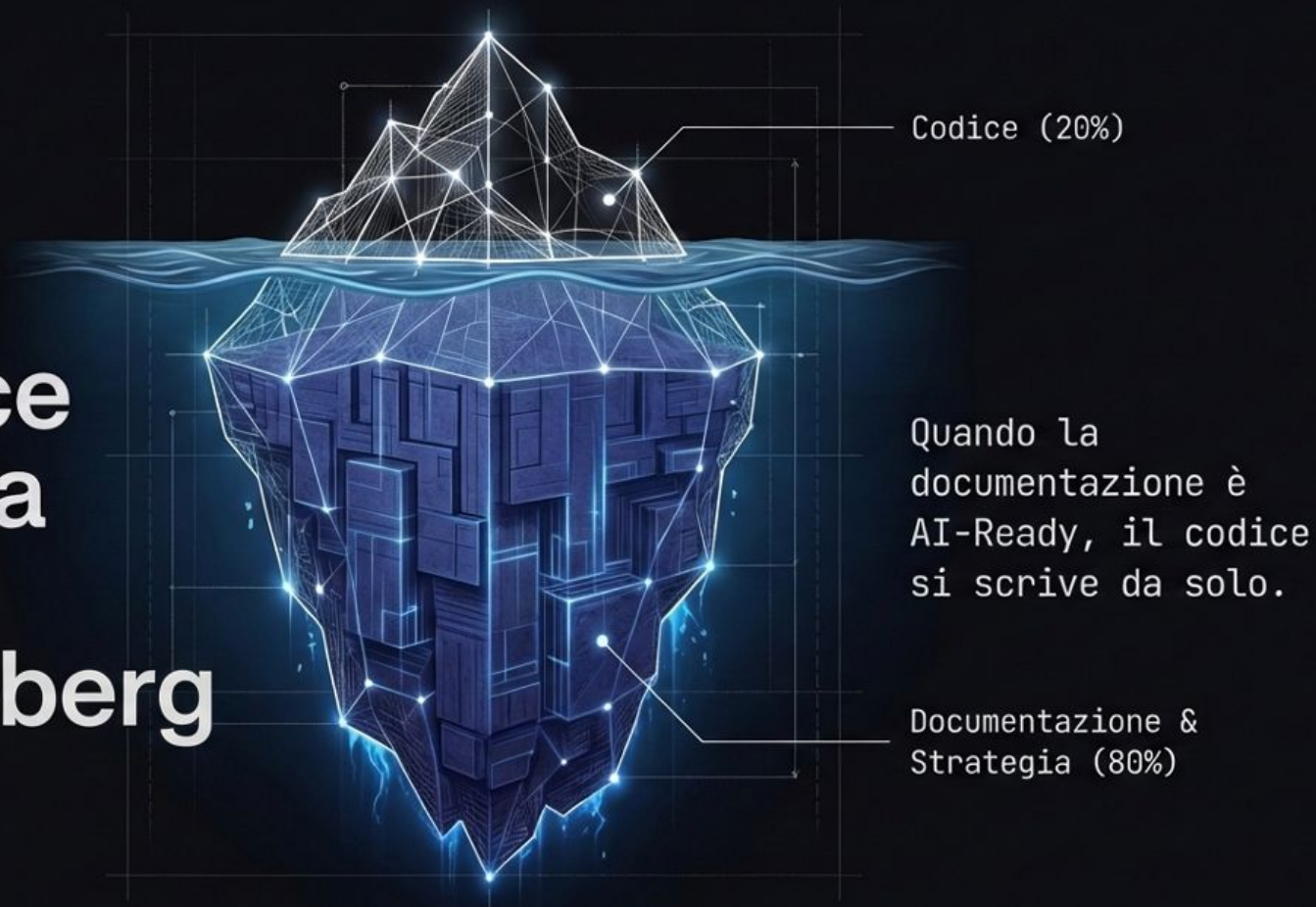
- È una “lavagna” utile a tracciare il lavoro rendendolo visibile a tutto il team





SCRUM STREAM CODING

Il Codice è solo la punta dell'iceberg



Miraggio della velocità

L'AI non riduce i tempi di consegna

- L'adozione di strumenti di Generative AI nel software development ha generato il cosiddetto “**Velocity Mirage**” (Miraggio della Velocità)
- Sebbene i singoli task di codifica appaiano accelerati, le tempistiche complessive del Product Development Life Cycle (PDLC) rimangono stazionarie o, paradossalmente, peggiorano
- I dati empirici dello Studio METR 2025 confermano questa distorsione: gli sviluppatori senior sono risultati il 19% più lenti nell'uso dell'IA, nonostante la percezione soggettiva fosse di un incremento di velocità del 20%
- Questa divergenza tra percezione e realtà nasce dal “Vibe Coding”
 - Un approccio esplorativo e non strutturato che produce codice rapido ma privo di coerenza architettonica
 - Senza una metodologia rigorosa, l'IA amplifica ogni ambiguità strategica



Analisi del Gap

Implementazione implicita

- Il superamento del Velocity Mirage richiede l'ancoraggio semantico dell'IA tramite specifiche
- Senza documenti di implementazione che vincolino l'output, l'IA opererà sul "maximum likelihood prior" dei dati di training, producendo soluzioni generiche e non ottimizzate per il dominio specifico

Fase	Sviluppo Tradizionale	Vibe Coding	Stream Coding
Pianificazione/ Architettura	Moderata (2-3 settimane)	Superficiale (Ore/Giorni)	Massima (40% del tempo)
Implementazione	Lenta e manuale	Istantanea (Task-level)	Automatica (15% del tempo)
Debugging/Refactoring	Proporzionale allo sviluppo	Elevatissimo (Effetto miraggio)	Quasi nullo



L'integrazione perfetta: il Telaio e il Motore

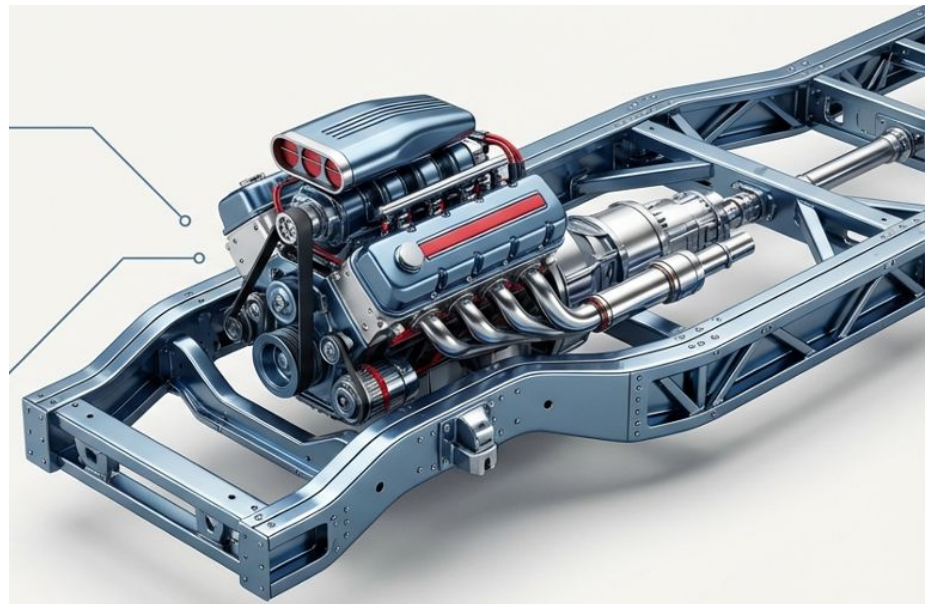
Non abbandonare le pratiche Agile. Potenziarne l'esecuzione

Il Telaio: SCRUM

- Fornisce la direzione strategica, la gestione delle priorità, i ruoli definiti e la sicurezza del processo
- Risponde alla domanda: **Cosa costruiamo e perchè?**

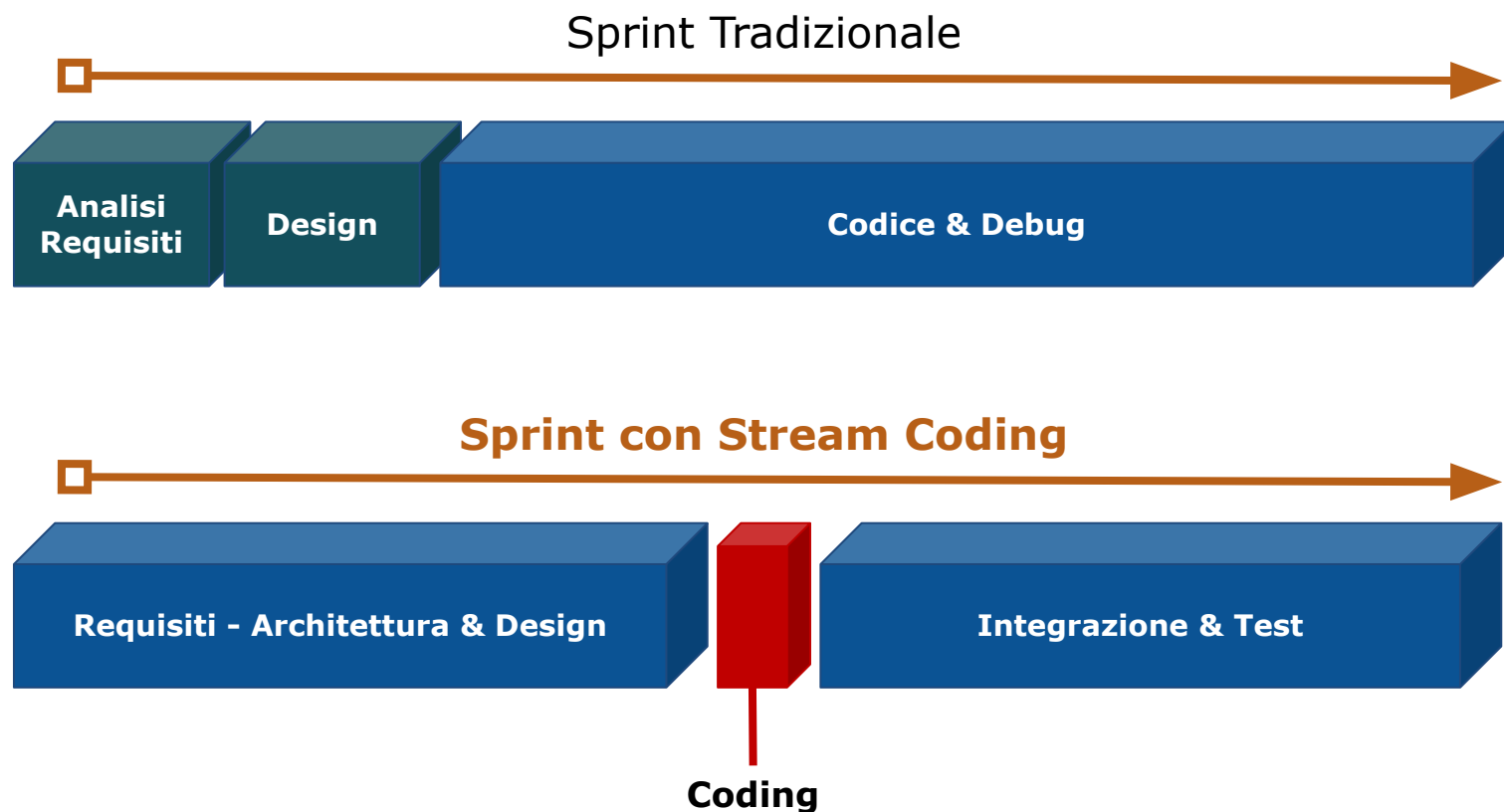
Il Motore: STREAM CODING

- Fornisce la velocità di esecuzione
- Trasforma gli elementi dello Sprint Backlog in codice funzionante a una velocità 10x grazie alla documentazione "AI-ready"
- Risponde alla domanda: **Come lo eseguiamo senza errori?**



Lo Sprint accelerato e il cambio di Focus

Riduzione del tempo di esecuzione del codice



Stream Coding:

La chiarezza architetturale come motore di esecuzione

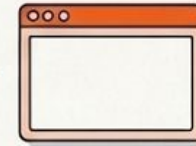
Ieri



Pensiero



Scrittura Codice
(Collo di Bottiglia)



Software

SCALE: 1:50

Oggi



Chiarezza Architetturale
(Collo di Bottiglia)



Generazione Istantanea IA



Software

SCALE: 1:50

Il Principio Fondamentale: Correggi la specifica, non il codice. Non si tratta di scrivere codice più velocemente, ma di creare specifiche così chiare che la generazione diventa automatica.

DIM: 16:9

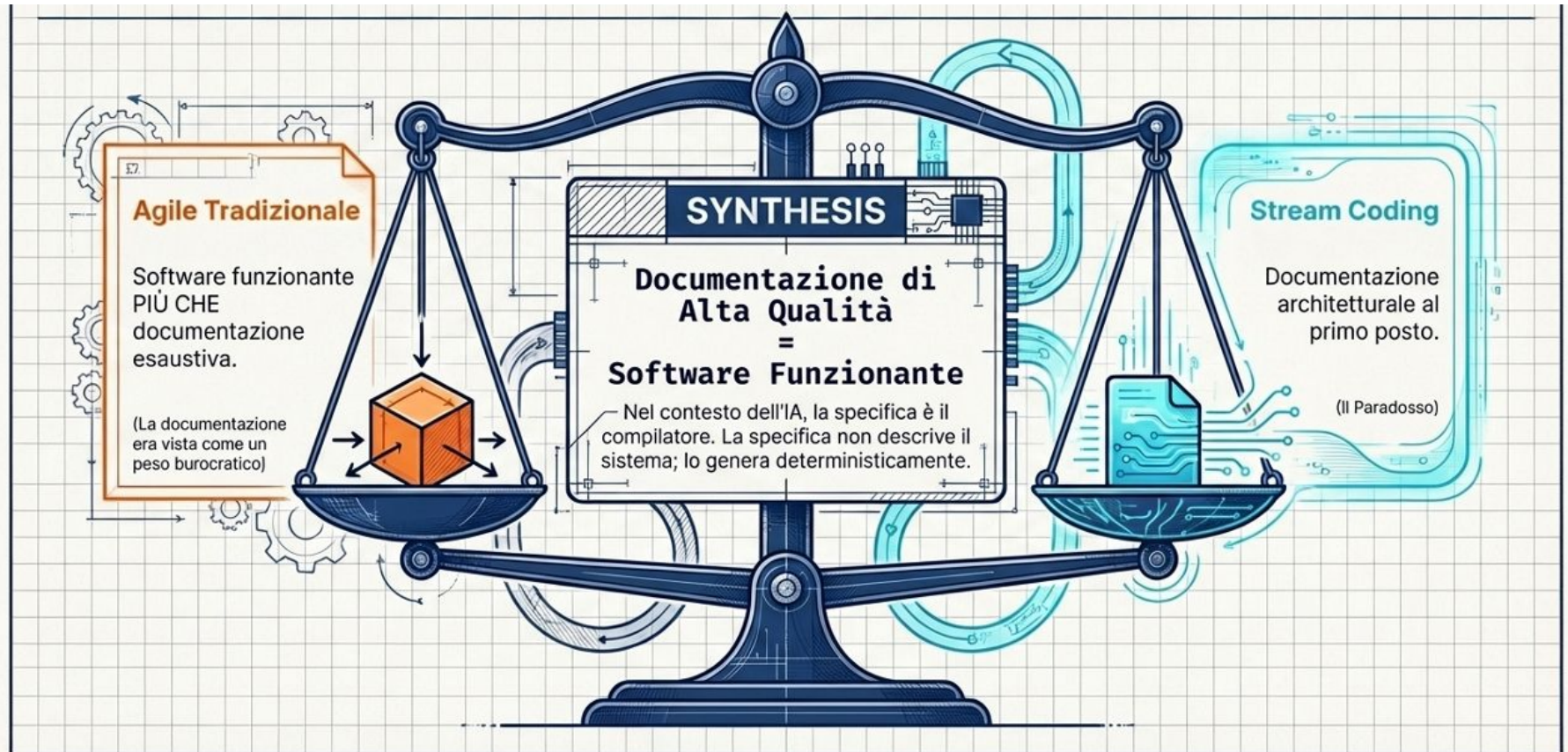
SCALE: 1:100

REF: STREAM-CODING-PRINCIPLE-V1



Stream Coding:

Risolvere il paradosso della documentazione nell'era dell'IA



Matrice di Trasformazione:

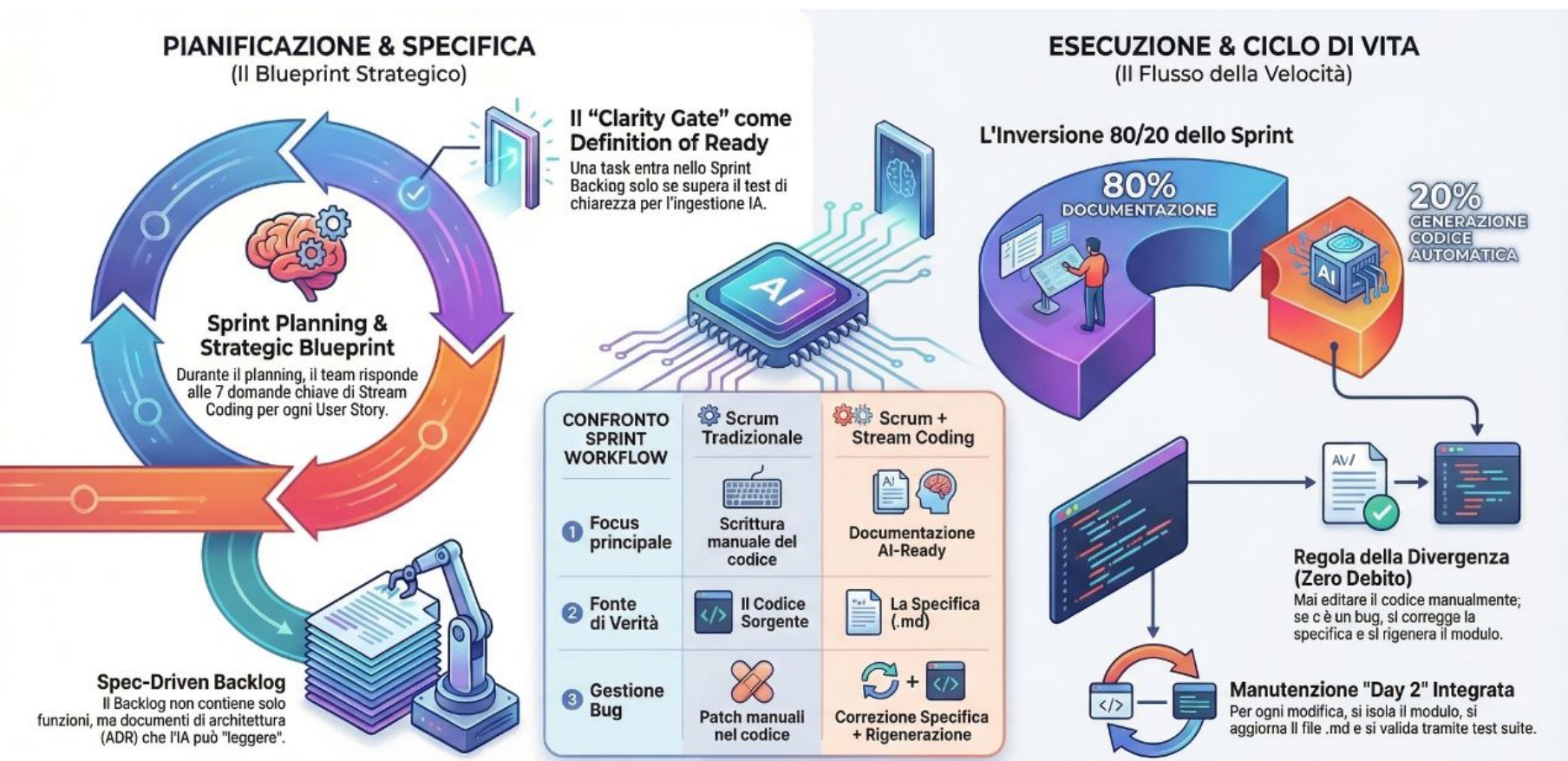
Da Scrum Tradizionale a AI-Accelerated Scrum

Dimensione	Scrum Tradizionale	Stream-Coded Scrum
Il Codice Base	È la fonte assoluta della verità.	È solo l'output compilato. La Specifica è la verità.
Sprint Planning	Stimare il tempo di progettazione e codifica.	Definire prompt architetturali e superare il Clarity Gate.
Daily Scrum	Quale codice ho scritto ieri?	Quale specifica ho chiarito ieri?
Il Collo di Bottiglia	Codifica/Sintassi.	Context window dell'IA / Chiarezza umana.



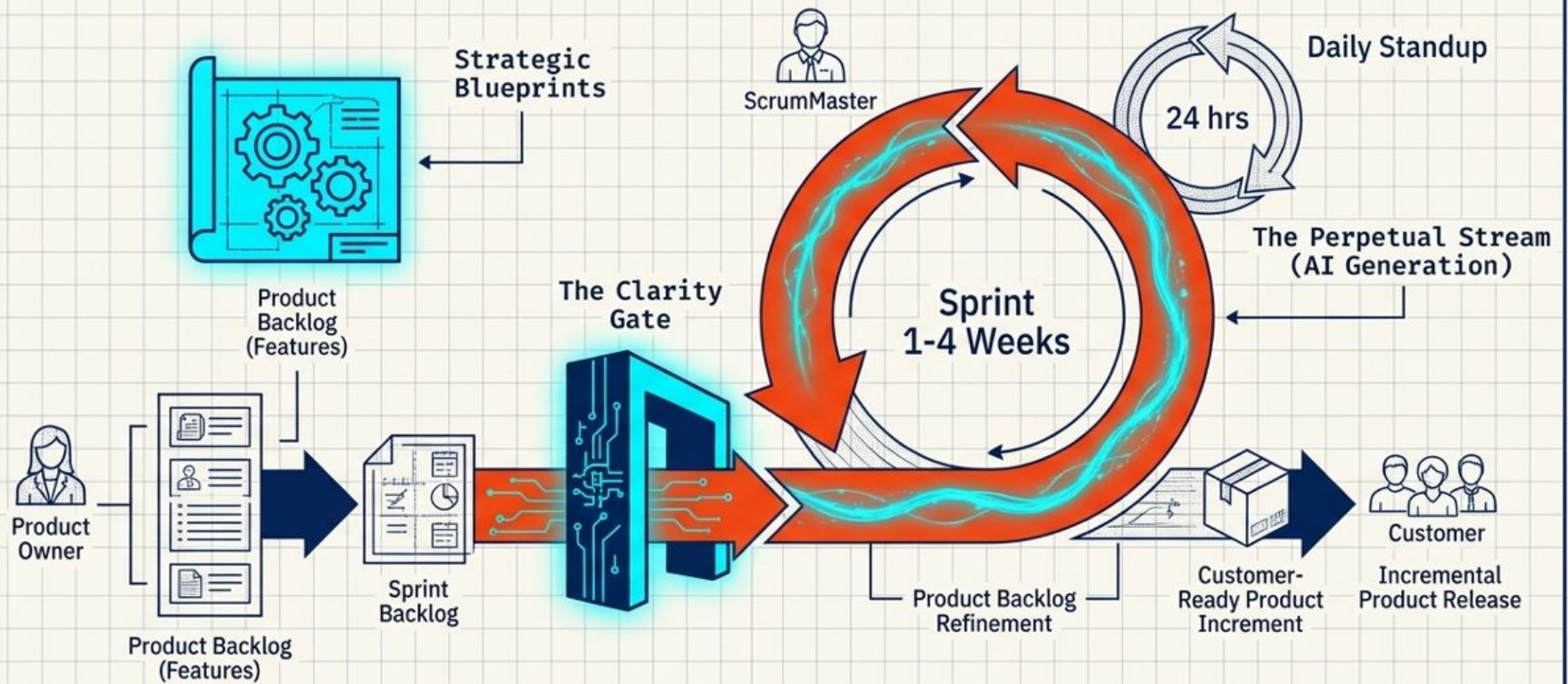
SCRUM STREAM CODING

Potenziare l'esecuzione dello SCRUM con il modello 40/20/20



SCRUM STREAM CODING

Il nuovo framework operativo: Stream Coding integrato nello Scrum

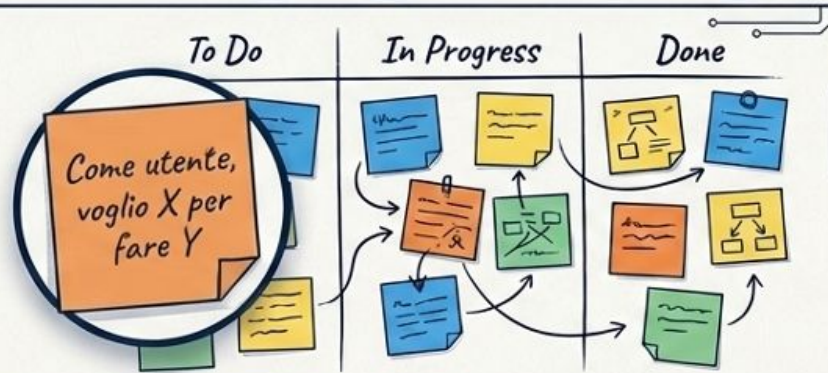


Stream Coding non sostituisce lo Scrum; ne diventa il motore di esecuzione interno.

Product Backlog & Refinement

Dalle User Story alle Strategic Blueprints

SCRUM



STREAM CODING

STRATEGIC BLUEPRINT

```
# Obiettivo
- Scopo principale della funzionalità e impatto sul sistema.

# Architettura dei Dati
- Schema JSON e relazioni tra entità.
...json
{
  "entity": "User",
  "fields": ["id", "role", "permissions"]
}
...

# Logica di Business
- Regole decisionali, flussi condizionali e validazioni.
IF condition THEN execute_action ELSE error_handling.
```

- Il Refinement trasforma idee vaghe in documenti pronti per l'IA.
- Ogni task deve rispondere a 7 domande strategiche prima di poter entrare nello Sprint.
- L'IA rifiuterà richieste vaghe e richiederà collegamenti documentali profondi.

Product Backlog & Refinement

Dalle User Story alle Strategic Blueprints

- Lo Stream Coding è una metodologia di documentazione, non di coding
- Il paradigma si sposta dal considerare il codice come l'asset principale al trattarlo come il "printout" (la stampa) di specifiche perfette
- Se la documentazione è AI-Ready, l'esecuzione diventa un processo deterministico e automatico

Strategic Blueprints: le 7 Domande

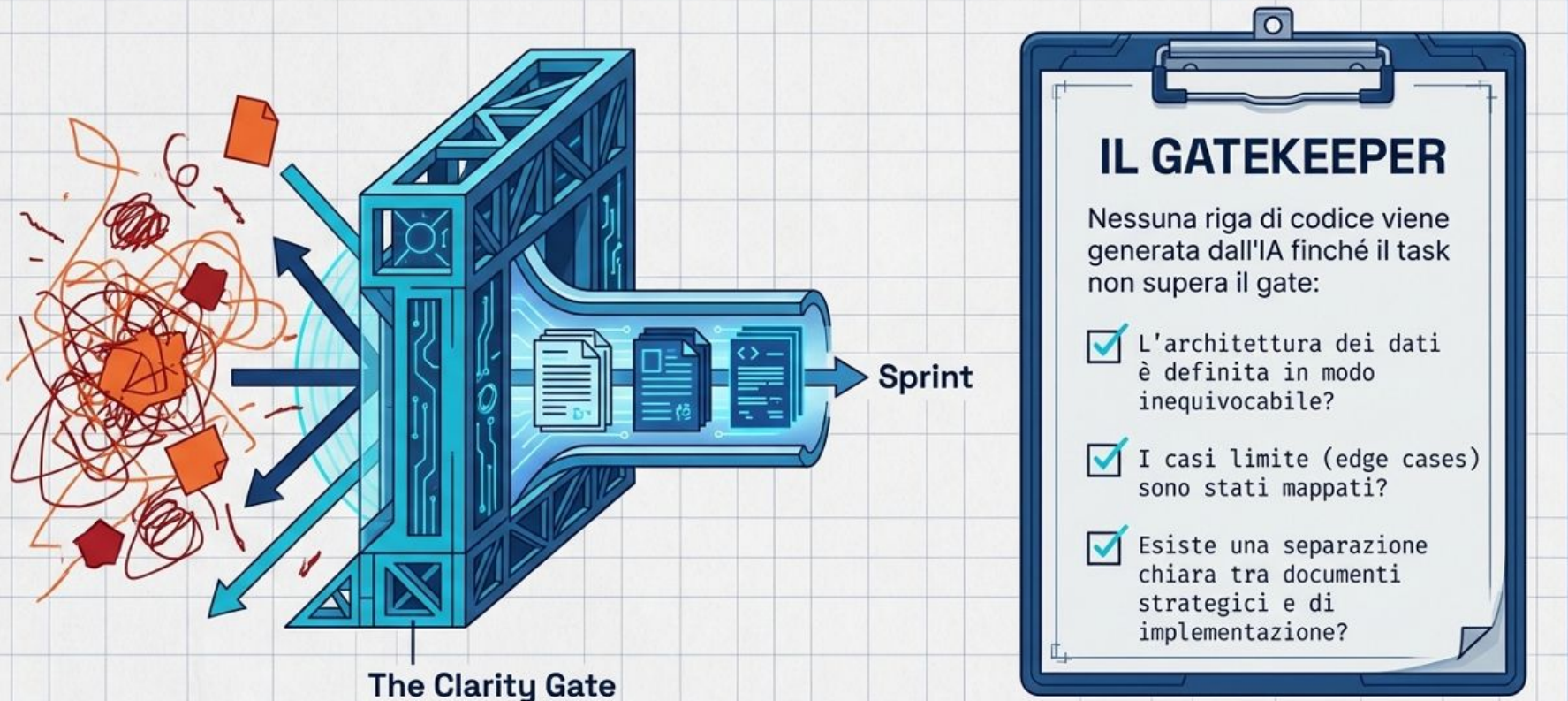
#	Domanda Strategica	Risposta "Vaga" (Reject)	Risposta "Specifica" (Require)
1	Quale problema risolvi?	"Aiutare gli utenti a gestire i task"	"[Persona] ottiene [Risultato] in [Contesto]"
2	Metriche di successo?	"Gli utenti risparmiano tempo"	"100 utenti, 25% conversione, 3 mesi"
3	Perché vincerai?	"Migliore UI/UX"	"Vantaggio strutturale: [Architettura/Dati]"
4	Decisione architettonica?	"Lasciamo decidere all'IA"	"Umano decide dopo analisi trade-off"
5	Razionale Tech Stack?	"Mi piace Node.js"	"Node: competenza team, velocità shipping"
6	Feature MVP?	10+ feature "must-have"	3-5 essenziali, il resto differito
7	Cosa NON stai costruendo?	"Vedremo in seguito"	"Esclusioni esplicite con rationale"



Clarity Gate e Sprint n.0

Controllo di qualità per poter procedere alla Pianificazione dello Sprint

Lo Sprint n.0 e l'istituzione del Clarity Gate



Il Clarity Gate e i requisiti Documentali

Controlli e Standard: la Checklist dei 13 punti

Foundation Checks

- [] **Actionability (Attuabilità):** Ogni sezione integra un dettaglio specifico di implementazione?
- [] **Currency (Aggiornamento):** Tutte le decisioni sono aggiornate?
- [] **Single Source of Truth (Singola fonte di verità):** Zero informazioni duplicate tra i documenti?
- [] **Decision, Not Wishes (Decisioni non Desideri):** Ogni affermazione è una decisione, non un desiderio?
- [] **Prompt-Ready (Pronto per il Prompt):** Inseriresti ogni sezione in un prompt?
- [] **No Future State (Nessun riferimento al futuro):** Rimosso ogni linguaggio al futuro ("faremo")?
- [] **No Fluff (Superfluo, Non Necessario):** Rimossi contenuti motivazionali o aspirazionali?

Il Clarity Gate e i requisiti Documentali

Controlli e Standard: la Checklist dei 13 punti

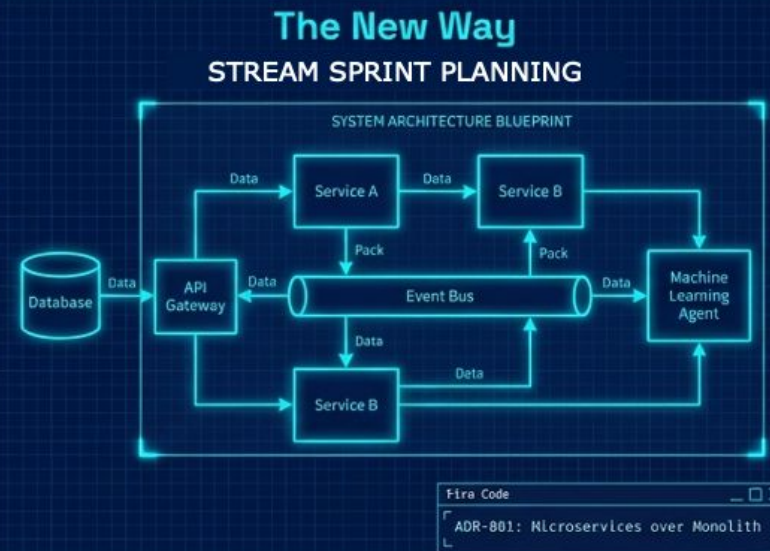
Document Architecture Checks

- ☐ **Document Type (Tipo di Documento):** Tipo documento identificato (Strategico, di Implementazione, o di Riferimento)?
- ☐ **Anti-patterns Placement (Posizionamento Anti-Pattern):** Presenti solo nei documenti di implementazione?
- ☐ **Test Cases Placement (Posizionamento dei casi di test):** Le specifiche di test sono presenti solo nella documentazione relativa all'implementazione e ai test?
- ☐ **Error Handling Placement (Posizionamento della gestione degli errori):** La Matrice degli errori è presente solo nella documentazione relativa all'implementazione?
- ☐ **Deep Links (Collegamenti Diretti):** Tutti i documenti contengono collegamenti diretti (e non semplici riferimenti generici)?
- ☐ **No Duplication (Nessuna Duplicazione):** I documenti strategici usano puntatori (link), non duplicati?

Pianificare l'architettura, non la sintassi



Stima basata sullo sforzo



L'Obiettivo Cambia

Da definire **"chi scriverà quale funzione"** a definire **"come interagiscono i moduli di sistema"**.

Il Ruolo del Team

Gli sviluppatori diventano **direttori d'orchestra**. Il **Planning** definisce i **prompt** per l'IA e stabilisce l'Architecture Decision Record (ADR).

Le Regole

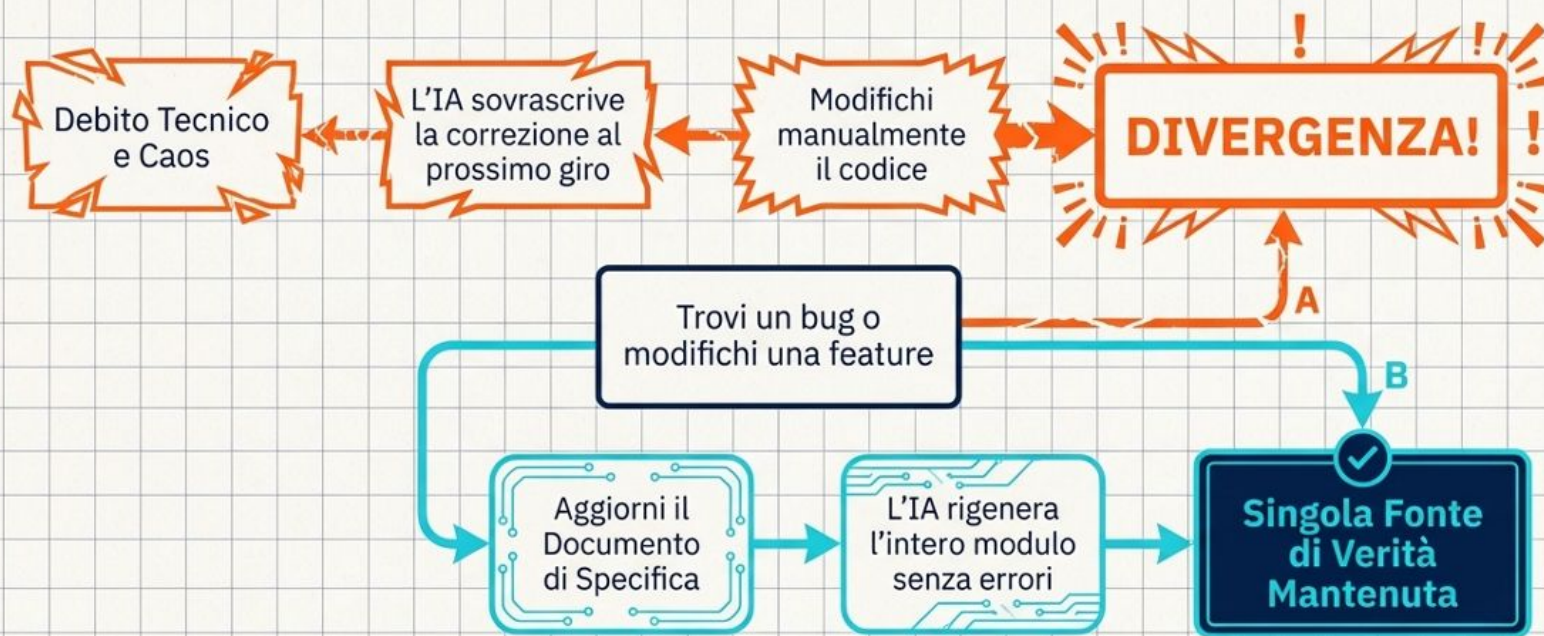
Il team concorda le **regole di sistema** (il "**Manifesto**") che l'agente IA dovrà seguire rigorosamente.



Sprint Execution

La regola della Divergenza

Sprint Execution: La Regola della Divergenza

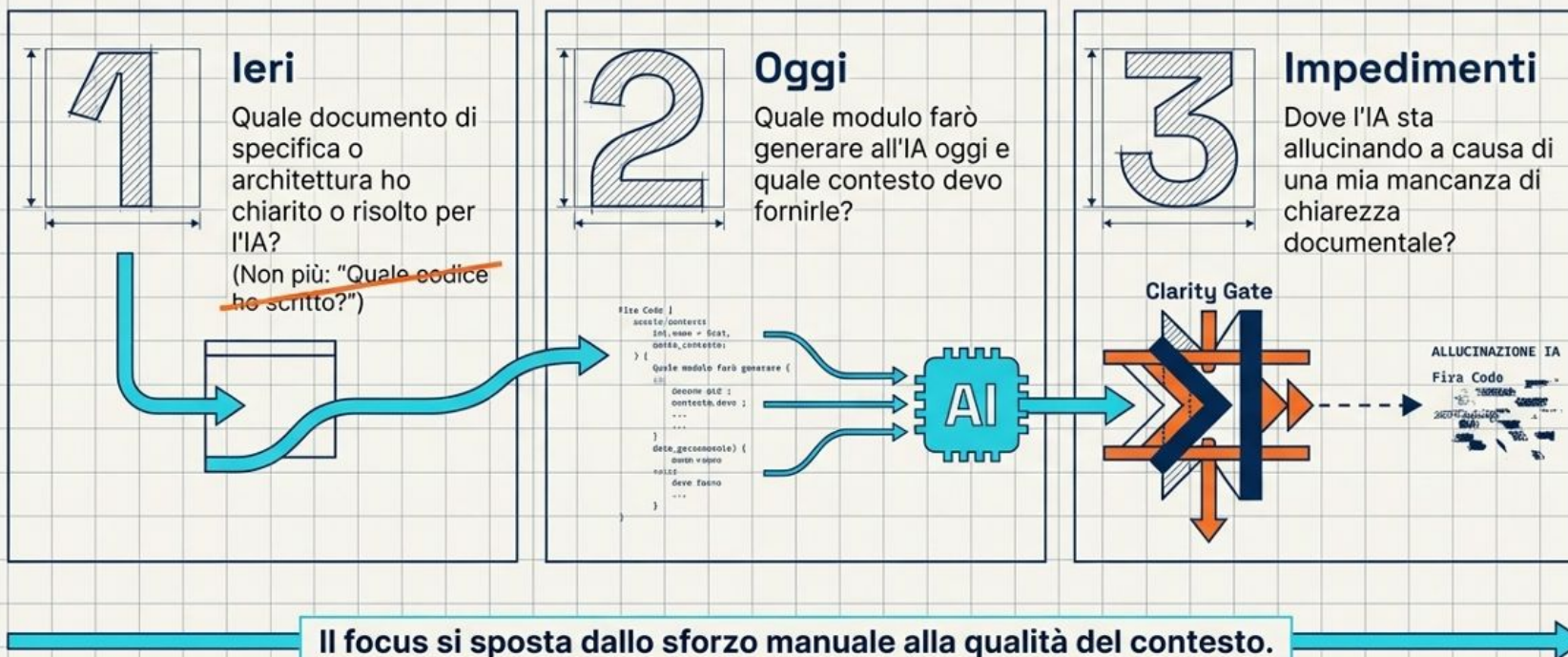


Ogni volta che modifichi il codice senza aggiornare la specifica, rompi il flusso.

Daily Scrum

in Stream Coding

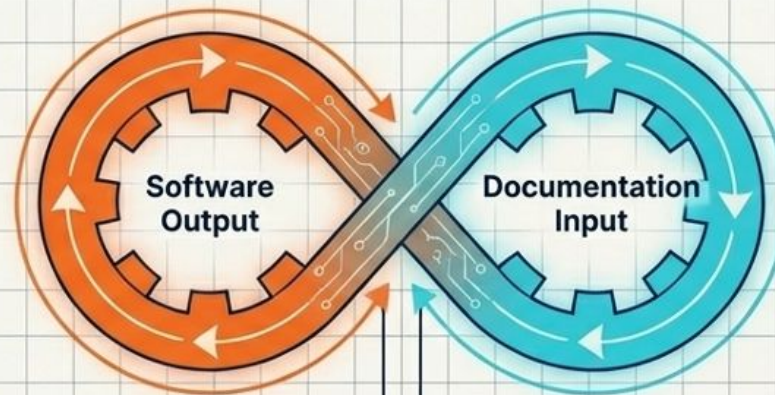
Daily Scrum 2.0: L'aggiornamento delle tre domande



Sprint Review & Sprint Retrospective

Valutare la Qualità del Prodotto e del Processo

- Lo Stream Coding si basa sull'esecuzione Single-Agent fornendo specifiche dense da permettere di operare senza la necessità di coordinamento complesso, eliminando i costi di latenza e instabilità



Sprint Review

Continuiamo a valutare il software funzionante con gli stakeholder.

Ma se c'è un errore, non incolpiamo il codice: analizziamo la specifica.

Sprint Retrospective

Il team valuta l'efficienza della comunicazione con l'IA.

"I nostri prompt erano troppo vaghi?"

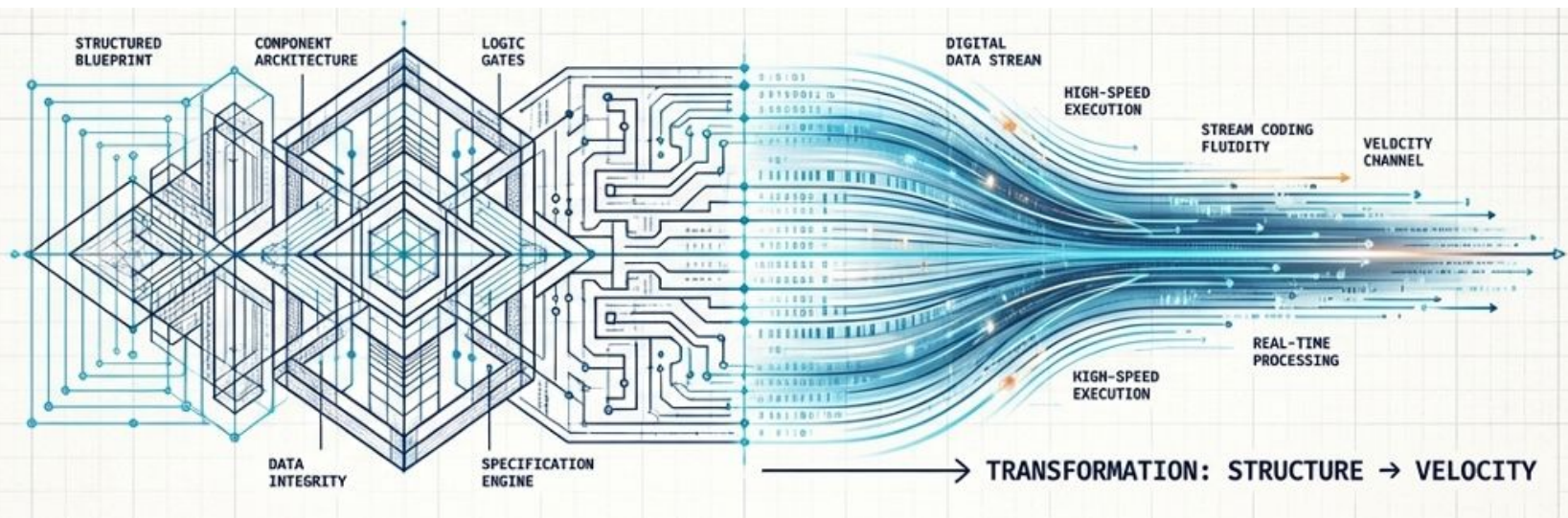
"Abbiamo creato divergenza modificando il codice a mano?"

Miglioramento Continuo = Affinamento dell'abilità di guidare l'agente IA.



SCRUM STREAM CODING

- L'integrazione dello Stream Coding nello SCRUM ne eleva l'astrazione
- L'ingegnere del Software si focalizza sulla progettazione del sistema software
- L'obiettivo è migliorare la qualità del prodotto e del processo



Riferimenti

- Ian Sommerville “Ingegneria del Software”, 10a ed. Pearson, 2017
 - Capitolo 3: Sviluppo Agile del Software
- Stream Coding:
<https://github.com/frmoretto/stream-coding>

Ulteriori fonti consigliate

- The Scrum Primer: scrumprimer.org
- Scrum.org: www.scrum.org

